

2026 상반기 커리큘럼 목차

AI/LLM & 데이터 엔지니어링.....	1
1. AI/LLM 기초	1
On-Premise Local LLM 구축 및 활용	1
AI를 활용한 로그 분석 및 해결	3
2. AI/LLM 인프라.....	5
Kubernetes 기반 폐쇄형 LLM 플랫폼 구축	5
3. MLOPs 통합.....	8
인프라 엔지니어를 위한 실전 MLOps 플랫폼 구축.....	8
리눅스 엔지니어링	11
1. 표준 리눅스(4일)	11
표준 리눅스 시스템 어드민	11
표준 리눅스 시스템 엔지니어	15
엔터프라이즈 리눅스 컨버전스 & 마이그레이션(RHEL·SUSE·Debian 계열 비교와 전환 실 습).....	18
2. 고급 리눅스(4일)	20
운영자를 위한 Bash 자동화 도구 제작.....	20
표준 리눅스 클러스터링 시스템(Pacemaker)	22
리눅스 트러블 슈팅.....	24
대규모 리눅스 노드 설치 및 구성 자동화	27
리눅스 하드닝(보안).....	29
컨테이너 & 쿠버네티스	30
1. 컨테이너 어드민	30
표준 포드만 컨테이너 어드민	30
2. 컨테이너 엔지니어.....	32
표준 쿠버네티스(컨테이너/가상화)	32
수세 컨테이너/가상화 클라우드 어드민	36
Okd 컨테이너 오케스트레이션(OpenShift 오픈소스 버전).....	38
클라우드 네이티브를 위한 MSA 구현	41
가상화 인프라.....	42
1. 가상화의 바닥을 이해하는 운영자 과정.....	42
2. KubeVirt 기반 Kubernetes 네이티브 가상머신 운영	44
3. OpenStack 서비스 설치(오픈스택 어드민).....	47
4. 쿠버네티스 기업용 가상화 하베스터 운영	50
자동화&Dev/MLOPs.....	52
1. 앤서블 코어 기반 인프라 자동화	52

2. OpenTofu(테라폼 오픈소스).....	54
3. AI Assisted Tekton CI/CD(CPU 기반 SLM을 활용한 YAML 생성과 자동화).....	56
퍼블릭 클라우드.....	59
1. AWS 기본 서비스	59
네트워크.....	60
1. OVS/OVN 기반 인프라 엔지니어 네트워크 실무 교육 커리큘럼	60
스토리지.....	62
2. 오픈소스 분산 스토리지 CEPH 기본.....	62
3. 분산 NAS GlusterFS 클러스터 구성 및 운영.....	64
언어.....	66
1. LLM 에이전트 작성을 위한 Go 언어 기초.....	66
2. 인프라 엔지니어를 위한 MicroJava(Quarkus) 기초	67
3. 인프라 엔지니어를 위한 파이썬 활용	69
모니터링.....	70
1. 오픈소스 통합 모니터링 운영 실무	72
세미나/부트캠프(이들 이하 교육).....	74
1. 하이브리드 클라우드를 위한 전략 부트캠프.....	74
2. 표준 CI/CD(테크톤) 구축 및 활용 부트캠프.....	74
3. Kube-virt/libvirtd 세미나	75
4. IaC기반 대규모 서비스 배포.....	76
5. IaC를 위한 AI 부트캠프.....	77
6. Podman 기반 LLM 서비스 부트캠프	77

AI/LLM & 데이터 엔지니어링

1. AI/LLM 기초

On-Premise Local LLM 구축 및 활용

교육명	On-Premise Local LLM 구축 및 활용
다이어그램	[Why] 보안/폐쇄망/비용/데이터 주권 [How] Local LLM (llama.cpp / OpenVINO) ↓ KIKI AI-Agent ↓ IaC (Ansible Core) [Result] Repeatable / Auditable / Safe Automation
수정	2026-02-06
다이어그램	[Student PC] - WEB EDITOR/SSH/kubectl [OpenStack Cluster] - Hosts VM for LLM + KIKI environment [Kubernetes/Podman Environment] - Pod: llm-infra - llama.cpp GGUF LLM Server (HTTP /v1) - OpenVINO GenAI LLM Server (OpenAI-compatible) - KIKI AI-Infra-Agent (FastAPI) - Shared Model Storage (GGUF / OV) - Pod: iaC-automation - Ansible Core environment - Generated Playbooks - Execution logs / reports Flow: Natural Language → KIKI CLI → LLM Engine → Playbook → Execution
교육개요	본 과정은 폐쇄형(Local) LLM을 기반으로 IaC(Infra as Code) 자동화 환경을 구축하고 운영하는 방법을 학습한다. llama.cpp(GGUF) 및 OpenVINO 기반 LLM 엔진을 구성하고, Hugging Face 모델을 활용하여 온프레미스 환경에서 독립적으로 운영 가능한 LLM 플랫폼을 설계한다. 이 과정은 Podman 기반 Pod 서비스 구성, KIKI AI-Infra-Agent 구축, 그리고 자연어 기반으로 Ansible Playbook 생성·검증·실행까지 자동화하는 실습 중심 교육이다. 학습자는 LLM을 활용한 운영 자동화 범위를 직접 실험하며, 조직 내 LLM 기반 운영 자동화 전략을 수립할 수 있는 실전 역량을 갖추게 된다.
목표	LLM 개념 및 구성 도구 이해 LLM과 컨테이너 기술(Podman, POD) 결합 필요성 이해

	IaC와 LLM의 관계 및 통합 방식 이해 다양한 IaC 도구(AWS CDK, Terraform, Ansible 등) 비교 LLM을 이용한 Ansible Playbook 자동 생성 및 검증 능력 확보
기간	4일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	폐쇄형 LLM 구축 및 구현을 원하는 시스템 관리자 여러 O/S 및 장치를 LLM기반으로 운영 및 관리를 원하는 시스템 관리자 IaC를 기반으로 시스템 통합을 원하는 사용자 IaC에 대한 지식이 없는 시스템 엔지니어
1일	LLM 개념 및 소개 - 폐쇄형 LLM과 공개형 LLM의 차이점 - 인공지능 영역에서 IaC의 역할 그리고 활용 방안 - AI 모델 기반으로 IaC 활용 및 자동화 방안 컨테이너와 LLM관계 그리고, POD 개념 - 왜? Pod를 사용해서 시스템을 구축해야 하는가? - 폐쇄형 LLM 시스템 구축 - 볼륨 디스크 및 GGUF 파일 다루기 - 간단한 API활용 - 간단한 에이전트 생성 및 앤서블 연동
2일	앤서블 기초 그리고 LLM - 앤서블 기본 문법 - 앤서블 기본 사용 방법 - 앤서블 인벤토리 및 실행 - LLM 기반으로 플레이북 생성 및 구성 - LLM 프롬프트와 플레이북 작성 조건 - 앤서블 직접 작성하기 - LLM 기반으로 플레이북 작성 및 비교
3일	플레이북 좀 더 활용하기 - 앤서블 플레이북 및 모듈 - 앤서블 코어 모듈 및 확장 모듈 - 앤서블에서 변수 개념 및 변수 선언 - LLM를 통한 모듈 확장 및 변수 확장
4/5일	플레이북 좀 더 활용하기 - 여러 서버에 서비스 배포 및 확인 - 시스템 상태 확인 및 보고서 자동 작성

AI를 활용한 로그 분석 및 해결

교육명	AI를 활용한 로그 분석 및 장애 원인 해결 실습
수정	2026-01-15
다이어그램	<pre> [System / App / K8s Logs] ↓ [File / Journal / API] ↓ [Pre-Processing] - Filter - Normalize - Chunk ↓ [LLM Engine] - Summarize - Classify - Explain ↓ [Insight] - Root Cause - Action Plan </pre>
교육개요	본 과정은 인프라 엔지니어가 대규모 로그를 AI(LLM)를 활용해 분석하고, 장애 원인 도출 및 해결 방향을 자동화·보조 판단 수준까지 끌어올리는 실습 중심 교육입니다. 단순 로그 수집이나 시각화가 아니라, • "왜 장애가 발생했는가?" • "이 로그 중 중요한 건 무엇인가?" • "조치 우선순위는 무엇인가?" 를 AI와 협업하여 더 빠르고 덜 틀리게 판단하는 방법에 대해서 강구합니다.
목표	수강생은 다음을 할 수 있게 됩니다. • 로그의 구조적 특징(system/app/k8s/audit) 이해 • 사람이 읽기 힘든 로그를 AI가 요약·분류·의미화 • 장애 시나리오 기반 원인 추론(Root Cause Analysis) • LLM 기반 로그 분석 자동화 파이프라인 설계 • 보안 로그/운영 로그에서 이상 징후 탐지
기간	총 2일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	인프라/클라우드/DevOps 엔지니어 Kubernetes/Linux 운영 담당자 로그 분석 및 해결책을 한 번에 해결책 제공받고 싶은 엔지니어 AI를 운영 도구로 활용하고 싶은 엔지니어

1일	로그와 장애의 본질 이해(로그는 왜 읽기 어려운가?) 로그의 종류와 성격 - system/kernel/app/audit/security 사람이 로그를 놓치는 이유 장애 분석의 전통적인 접근 방식 인프라 엔지니어의 LLM 그리고 AI 서비스 - 가상머신/컨테이너 기반으로 서비스 아키텍처 - GPU/CPU 차이점. CPU기반으로 구성 시 주의할 부분 - 내부용 LLM 서비스 구성 - LLM 서비스를 위한 하드웨어 구성 - 왜 GPU기반의 설계는 과잉을 초래 하는가? AI가 이해할 수 있는 로그 만들기 로그파일은 무엇인가? - 왜 AI는 일반 로그파일 분석이 어려운가? - 텍스트 파일과 그리고 로그 전처리 - 덩어리(chunking)기반으로 분리 분석 - 시간개념과 데이터 처리 로그파일 기반으로 전처리 활용 LLM 입력 한계와 전략 - LLM문자와 그리고 데이터셋 처리(TEXT TO JSON) TEXT TO SEMANTIC BLOCK 개념 및 구성 - JSON만 가능한가? 다른 형식 및 유형은 불가능한가? 텍스트 로그파일을 키-값 형태로 전환
----	---

2일	AI에게 로그 분석을 시킨다 로그분석 활용 - 로그 분석에 적합한 LLM 활용 방식 - 요약/분류/원인 추론 차이 - 잘못된 AI 판단을 구분하는 법(AI Hallucination 방지) 로그분석 수행 - 장애 로그 자동 요약 - 에러 패턴 분류 "이 로그에서 이상한 점은?" 질문 실습 - 사람 vs AI 판단 비교 자동화와 RAG 기반 분석 RAG와 LANGCHAIN의 차이점 - 인프라에서는 왜 RAG를 더 많이 활용하는가? - langchain의 장단점 - 에이전트가 필요한가? 에이전트가 필요한 이유. - 간단한 에이전트 작성 로그분석 시작 - 로그 분석을 위한 로그 변환 - 변환된 로그 기반으로 로그 분석 시작 - 기능 및 기술 리뷰 장애분석 자동화 LLM과 자동화 - LLM에서 왜 자동화가 필요한가? - 프롬프트 자동화 VS IaC - 오픈소스에서 사용하는 대표적인 IaC도구 - 앤서블 기반으로 장애처리 자동화 적용 - 로그분석 및 장애처리 자동화 - 시스템 상태로 기록 LLM으로 확인
----	---

2. AI/LLM 인프라

Kubernetes 기반 폐쇄형 LLM 플랫폼 구축

교육명	OpenStack & Kubernetes 기반 폐쇄형 LLM 플랫폼 구축
권장	On-Premise Local LLM 기반 IaC 자동화 실습
수정	
다이어그램	<pre> [Kubernetes Cluster] ├── Namespace: llm-cpu │ ├── llama.cpp Server │ └── OpenVINO GenAI Server </pre>

	└─ vLLM (CPU mode) └─ Namespace: llm-gpu └─ vLLM/TGI (GPU) └─ Namespace: gateway └─ API/LoadBalancer/simple UI
교육개요	본 과정은 Kubernetes를 중심으로 폐쇄형 LLM(Closed LLM) 플랫폼을 설계·구축·운영하는 방법을 학습하는 실무 중심 교육입니다. CPU 및 GPU 자원을 보유한 온프레미스 환경에서 Kubernetes 클러스터를 직접 구성하고, 그 위에 CPU 기반/GPU 기반 LLM 추론 서비스를 컨테이너 형태로 배포합니다. 교육의 마지막 단계에서는 • 프롬프트 기반 LLM API 호출 • CPU/GPU 역할 분리 전략 • 폐쇄망 환경에서의 LLM 운영 아키텍처 를 종합적으로 정리합니다.
목표	Kubernetes 기반 LLM 플랫폼 아키텍처 이해 CPU/GPU 노드가 혼합된 Kubernetes 클러스터 설계 CPU 기반 LLM 서비스와 GPU 기반 고성능 LLM 서비스 배포 LLM REST API 구조 이해 및 호출 실습 폐쇄형 LLM 시스템의 운영 전략과 확장 방향 정리
기간	5일, 총 40시간 (1일 8시간)
랩	온프레미스 또는 프라이빗 환경 Kubernetes 클러스터 • Control Plane: 1~3대 (CPU) • Worker Node: ○ CPU 노드 ○ GPU 노드 (가능한 경우) GPU: Tesla T4, RTX 30xx 등 (환경에 따라) • GPU 미보유 시 CPU 기반으로 대체 실습 가능 네트워크 • 관리망 (K8s 노드 관리) • LLM 내부 통신망 • 외부 접근용 (API/LB) 스토리지 • NFS 또는 Kubernetes PV 기반 공유 스토리지
교육대상	폐쇄형 LLM 시스템을 직접 구축·운영하고자 하는 시스템/인프라 엔지니어 OpenStack과 Kubernetes 환경에서 AI/LLM 워크로드를 올려보고 싶은 실무자 GPU 자원을 활용한 LLM/AI 인프라 설계에 관심 있는 엔지니어
1일	LLM 개요

	• LLM 구조와 추론(Inference) 개념 • 파라미터 수, 모델 크기, 컨텍스트 길이와 자원 요구량 • 폐쇄/공개형 LLM 차이점 ◦ 보안 ◦ 데이터 운영 및 관리 ◦ 런타임 및 모델 운영 관점 비교 ◦ 과잉 GPU 설계의 위험도 Kubernetes와 LLM • 왜 LLM 서비스를 Kubernetes 위에 올리는가? • 컨테이너 기반 LLM의 장점 • 단일 LLM 서버 vs LLM 플랫폼 구조 Kubernetes 기반 LLM 아키텍처 설계 • Namespace 분리 전략 • CPU 추론 영역/GPU 추론 영역 분리 • Gateway(API) 계층의 역할
2일	Kubernetes 핵심 개념 정리 • Pod/Deployment/Service/Ingress(API GATEWAY) • ConfigMap/Secret • Resource Request/Limit 클러스터 구성 전략 • Control Plane/Compute 분리 • 단일 클러스터에서 CPU/GPU 혼합 운영 노드 역할 분리 • CPU 노드 • GPU 노드 • Node Label • Taint/Toleration 개념과 실습
3일	CPU 기반 LLM 엔진 이해 • llama.cpp • OpenVINO GenAI • vLLM (CPU mode) • 모델 포맷: gguf 등 CPU 추론의 특징 • 비용 효율 • 성능 한계 • 개발/테스트/내부 서비스에 적합한 이유 LLM API 설계 • REST API 기본 구조 ◦ /v1/completions

	○ /v1/chat/completions • 요청/응답 구조 • 토큰 제한, 타임아웃, 동시 요청 고려사항
4일	Kubernetes에서 GPU 사용 구조 • GPU Device Plugin 개념 • nvidia.com/gpu 리소스 이해 GPU 노드 설계 • GPU 전용 노드 Label • 특정 Namespace 또는 Deployment만 GPU 사용하도록 제한 고성능 LLM 서비스 • vLLM/TGI 기반 GPU 추론 • 소형모델(8B미만) 개념 소개 • 대형 모델(13B~70B) 개념 소개 • 긴 컨텍스트, 처리량 중심 워크로드 설계
5일	전체 구조 정리 • Kubernetes 클러스터 • CPU LLM Inference Pod • GPU LLM Inference Pod • Gateway/API/간단 UI CPU/GPU 운영 전략 • 요청 유형에 따른 자원 분기 ○ 실험/개발/테스트 → CPU ○ 고성능/실서비스 → GPU • 비용과 성능의 균형 설계 확장 시나리오 • 여러 LLM 모델 동시 운영 • 모델별 엔드포인트 분리 • RAG 및 Vector DB 연계 (개념 수준) • 사내 서비스/API와의 연동 방향

3. MLOPs 통합

인프라 엔지니어를 위한 실전 MLOps 플랫폼 구축

교육명	인프라 엔지니어를 위한 실전 MLOps 플랫폼 구축
수정	
다이어그램	[Student PC] - Browser/k8s/kubectl <pre> +-----+ OpenStack Cluster (Controller / Compute / Storage) +-----+ </pre>

	↓ v <pre> +=====+ Kubernetes Cluster (Single) RKE2 or K8s on OpenStack VM +=====+ ns: llm-infra - Option 1) llama.cpp GGUF Server (HTTP /v1) - Option 2) OpenVINO GenAI LLM Server (OpenAI compatible) - KIKI AI-Infra-Agent (FastAPI) - VectorDB (Qdrant / pgvector) ns: rag-ml - RAG Indexer (Docs → Split → Embed → Job/Pod) - Tekton Pipelines - Kubeflow Pipelines (minimal components) </pre>
교육개요	본 과정은 인프라 엔지니어 관점에서 MLOps 전체 흐름을 이해하고 직접 구축하는 실습 중심 교육이다. 모델 알고리즘을 깊게 다루기보다, • 실시간 데이터 스트리밍 • Feature/ML 파이프라인 • 작은 LLM 기반 추론 서비스 • RAG 및 파이프라인 자동화 • Kubernetes 기반 운영 구조 를 End-to-End로 연결하는 인프라 설계 및 운영 역량에 초점을 둔다. Kafka-Spark/Flink-LLM-Pipeline(Kubeflow/FuseML)을 단계적으로 통합하며, GPU 없이도 실제 기업 환경에서 적용 가능한 현실적인 MLOps 최소 단위를 완성한다.
목표	- 인프라 엔지니어 관점의 MLOps 전체 아키텍처 이해 - 실시간 데이터 스트리밍 기반 ML/LLM 파이프라인 구축 - 작은 LLM(1.5B~8B) 기반 추론 서비스 운영 - RAG(Vector DB) 기반 LLM 품질 향상 구조 이해 - Tekton/Kubeflow/FuseML 기반 파이프라인 자동화 - Kafka Replay를 활용한 재처리·재추론 운영 전략 습득 - 최종적으로 운영 가능한 MLOps Mini-Platform 완성
기간	5일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	인프라 엔지니어 DevOps/SRE/Platform 엔지니어 Kubernetes 기반 플랫폼 운영자 AI/ML을 운영 관점에서 이해해야 하는 엔지니어 GPU 없이 AI-LLM 도입을 검토하는 기업/기관
1일	MLOps 전체 구조 이해 (Infra 관점) 이론 • Modern MLOps 아키텍처 개요

	• Streaming → ML → LLM → Pipeline 흐름 • 데이터 엔지니어링 vs 인프라 엔지니어 역할 구분 • Batch 처리와 Streaming 처리 차이 • 작은 LLM을 사용하는 이유 (비용·운영 관점) 실습 • 실습 환경 VM 준비 • Podman 또는 Kubernetes 기본 구성 • Kafka 단일 브로커 설치 • 샘플 로그 데이터 스트림 구성 • 전체 파이프라인 데이터 흐름 시각화
2일	Kafka 기반 실시간 Feature 파이프라인 이론 • Kafka 아키텍처 ◦ Broker / Topic / Partition / Offset • Fault-tolerant Streaming 구조 • Spark vs Flink 역할 비교 • Feature Engineering 개념 (Infra 관점) 실습 • Kafka Topic 설계 ◦ raw-logs ◦ features ◦ llm-requests • Spark ETL 실습 (Batch) • Flink Streaming 실습 (실시간) • Kafka → Spark/Flink → Feature Topic 구성
3일	작은 LLM + RAG 통합 이론 • Small LLM 구조 (1.5B~8B) • llama.cpp 서버 모드 구조 • LLM API 패턴 (비동기 처리) • RAG 아키텍처 ◦ Document Split ◦ Embedding ◦ Vector Search 실습 • llama.cpp 서버 컨테이너 실행 • Feature 데이터 기반 LLM 요약/분류 • 문서 전처리 및 Chunk 생성 • Embedding 모델 생성 (CPU)

	- Vector DB(Qdrant/pgvector) 구축 - RAG 기반 질의 응답 테스트
4일	Pipeline 기반 자동화(Tekton/Kubeflow/FuseML) 이론 - Pipeline이 필요한 이유 - Tekton vs Kubeflow vs FuseML 비교 - Infra 엔지니어 시점의 선택 기준 - 온프레미스/폐쇄망 운영 전략 실습 - Tekton 기반 RAG 인덱싱 Pipeline - 문서 수집 → 전처리 → Embedding → 저장 - Kubeflow 또는 FuseML 기반 ML Pipeline - 학습 → 평가 → 등록 - Pipeline 재실행 및 실패 처리
5일	운영형 MLOps 완성 이론 - End-to-End 운영 아키텍처 - Kafka Replay 기반 재처리·재추론 - LLM Worker 수평 확장 전략 - 모델 버전 관리 개념 - 모니터링 및 장애 대응 실습 (최종 PoC) - 전체 통합 실습: - 로그 수집 - 실시간 Feature 생성 - ML 분석 - RAG + LLM 설명 생성 - Pipeline 자동 실행 - Grafana 기반 모니터링 - 실제 기업 적용 시나리오 정리 - 아키텍처 리뷰 & Q&A

리눅스 엔지니어링

1. 표준 리눅스(4일)

표준 리눅스 시스템 어드민

교육명	표준 리눅스 시스템 어드민		
수정	2026-01-02		
다이아그램		Hardware	

	<pre> graph TD A["CPU / Mem / Storage / NIC"] --> B["Linux Kernel syscalls / scheduler / cgroups / namespaces"] B --> C["systemd (PID 1)"] B --> D["Network Stack (NM / networkd / nft)"] B --> E["Storage Stack (DM / LVM / Stratis)"] C --> F["systemd Units services / sockets timers / targets"] D --> G["ip / ss / nftables networkctl netlink API"] E --> H["LVM / Stratis XFS/Btrfs VDO/ZRAM"] F --> I["Journald (Binary Log Store)"] G --> J["Logind / Session (User session mgmt)"] H --> J I --> K["journalctl filtering / boot"] J --> L["loginctl user mgmt / seat"] K --> M["Application Layer Podman / systemd-nspawn / httpd / nginx / DB / custom svc"] L --> M </pre>
교육개요	본 과정은 현대 리눅스 시스템이 systemd 기반으로 완전히 통합된 구조를 중심으로 운영되는 흐름을 이해하고, 그 기반 위에서 서버 운영·스토리지·네트워크·보안·자동화 기술을 학습하는 실무 과정이다. 리눅스의 문화적 배경, 파일시스템 표준(FHS/LSB) 변화, systemd 전환 이유, 현대적 네트워크 구성, 성능 모니터링, 파일시스템, LVM/Btrfs/XFS, 컨테이너(Podman) 까지 현대 리눅스 운영자에게 반드시 필요한 필수 주제를 체계적으로 정리한다. 단순 명령어 나열식의 기초 과정이 아니라, 현대 리눅스 운영의 표준 패턴과 전체 아키텍처를 익히는 실무 중심 커리큘럼이다.
목표	systemd 기반 현대 리눅스 운영 구조(systemd units, journald, networkd 등) 이해 리눅스 표준 디렉터리(FHS/LSB)와 배포판 차이 이해 systemd 기반 서비스/로그/네트워크/타임/호스트 관리 능력 확보 네트워크, 사용자, 프로세스, 패키지 관리 심화 최신 스토리지 기술(LVM2, Btrfs, XFS, Stratis, VDO) 구성 능력 확보 성능 모니터링(iostat, vmstat, ss, atop) 실무 능력 강화 systemd 기반 커널/모듈/udev 연동 구조 이해 Podman 기반 컨테이너 운영 기본 실습 자동화를 위한 YAML/TOML/Ansible 기초 습득

기간	총 4일, 3일로 조정가능 과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	1년 이상 혹은 미만의 리눅스 관련 시스템 관리자 혹은 네트워크 엔지니어 가상화 혹은 컨테이너 관련 프로젝트 시스템 엔지니어 혹은 소프트웨어 엔지니어
1일	리눅스와 오픈소스 관계 리눅스 쉘 및 커널의 구성 - 리눅스 표준 디렉터리 구성 및 역할 설명 - FSH VS LSB의 차이점 - 왜 리눅스 배포판은 디렉터리가 다른가? systemd기반으로 통한 시스템 블록 이해 - systemd와 system-v의 차이점 - 왜 대다수 리눅스 배포판은 systemd로 전환하였는가? - systemd에서 제공하는 주요 리소스 유형 - 앞으로 시스템 자원 유닛(/etc/fstab, block device) 서비스 및 로그 관리를 위한 명령어(로깅 분석) - rsyslog에서 로그 관리 및 rsyslog의 미래 - systemd, journald기반으로 시스템 블록 로깅 분석 - systemd기반으로 중앙 로깅 시스템 구성 자주 사용하는 기본적인 리눅스 명령어 - systemd-booted기반으로 부팅 관리 - systemd-kernel 관리 - systemd-locale, loginctl - systemd-machine-id 리눅스 커널 동작 구조(간단하게) - 현재 리눅스 커널의 아키텍처 및 기능
2일	새로운 네트워크 구성 및 관리자 사용 - systemd-network - NetworkManager - ip, ss, route - rh-ifcfg-scripts - systemd-resolved, systemd-hostnamed - systemd-timesyncd, chronyd 새로운 방화벽 시스템(nftables, firewalld) - netfilters는 무엇인가? - iptables vs nftables - firewalld와 nftables의 관계 - firewalld-cmd명령어 사용 방법 - nft명령어 사용 방법

	프로세스 및 사용자 관리를 위한 명령어 - 사용자 생성 및 비밀번호 설정 - 사용자 관리를 위한 usermod - 특정 사용자 그룹 생성 및 설정 - 그룹 관리를 위한 groupmod - 패스워드 생성 - 시스템 프로세스 및 사용자 프로세스 차이점 - 프로세스 관리를 위한 명령어 파일 시스템 퍼미션 - 사용자/그룹 및 시스템 퍼미션 - ACL기반 퍼미션 관리 - 현재 리눅스에서 umask사용 및 관리 방법 패키지 관리자 - 리눅스 배포판에서 제공하는 패키지(RPM, DEB) - 고급 패키지 관리자(DNF/YUM/APT) - 리눅스 커널 패키지 관리(소스코드 및 바이너리) - - 패키지 리 빌드는 필요한가? 왜 법적으로 문제가 발생하는가?
3일	LVM개념 및 구성 및 구현 - LVM2의 개념 - 왜 레드햇 계열은 여전히 LVM2를 사용하는가? - LVM2와 파티션 영역 관계 - PV/VG/LV생성 및 관리 - LVM2의 확장 기능 사용 - 백업 및 복구 BTRFS - BTRFS 파일 시스템 소개 및 구성 - Pool 생성 및 관리 - Volume 생성 및 관리 - 파일 시스템 수정 도구 XFS - XFS 파일 시스템 소개 및 구성 - XFS 관리 도구 - Stratis for XFS 설명 및 구성 - VDO for XFS 설명 및 구성 - 파일 시스템 수정도구 성능 모니터링 - systemd기반에서 시스템 모니터링 - sysstat와 iostat, vmstat

	- 네트워크 모니터링 방법
4일	리눅스 커널과 모듈 - 리눅스 커널에서 모듈의 역할 - 모듈 튜닝 및 설정 - 모듈 디버깅 - 커널 파라미터 조정하기(tuned, systemd-sysctl) - udev와 그리고 dbus의 관계 백업 도구 및 응급 복구 - 리눅스에서 제공하는 백업 방법 및 도구(tar, rsync, dd) - 이미지 기반 백업 도구(disk, partition) - 응급 복구 - single, emergency mode 가상머신 및 컨테이너 기술 소개 - systemd-machine - libvirt, libguest-tools - podman과 그리고 conmon, crun - podman기반으로 pod, container 생성 - buildah, skopeo를 통한 이미지 구성 및 이미지 서버 구성 시스템 자동화 - ansible/terraform/salt 비교 - 각 도구별 자동화 예제 및 구성

표준 리눅스 시스템 엔지니어

교육명	표준 리눅스 시스템 엔지니어
수정	2025년 12월 02일
교육개요	표준 리눅스 시스템 어드민 과정은 현대 리눅스 운영 환경에서 핵심이 된 systemd 중심 구조, journald 기반 로깅, Stratis/LVM 기반 스토리지 계층, 현대 네트워크 스택(ip/nftables) 등을 종합적으로 학습하는 과정입니다. 기존 SysV 기반 방식(ifconfig, netstat, syslog 등)에서 완전히 탈피하여, 현대 Linux가 제공하는 서비스 관리/세션 관리/네트워크 구성/스토리지 계층/성능 분석/장애 대응을 랩 기반으로 학습합니다. OpenStack 기반 VM에서 실습을 진행하며 실제 운영 환경과 동일한 구조에서 안정적인 인프라 운영 역량을 확보하도록 설계되었습니다.
목표	현대 Linux(systemd 기반)의 통합 구조를 체계적으로 이해 systemctl, loginctl, journalctl 등 통합 관리 명령 완전 습득 journald 기반 로그 구조·필터링·중앙화 이해 Stratis / LVM / XFS 기반 최신 파일시스템 계층 이해 ip, ss, nftables 중심의 현대 네트워크 구성을 실습

	성능/장애 분석 및 문제 해결 능력 향상 systemd 유닛 작성 및 운영 자동화 기초 확보
기간	총 4일 교육
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	기존 리눅스 관리 경험자 현대 Linux 시스템 구조(systemd 기반)를 재정비하려는 엔지니어 인프라/클라우드 운영자 DevOps-Kubernetes 이전 단계에서 필수 리눅스 운영 역량이 필요한 실무자
1일	현대 Linux 아키텍처 완전 이해 (systemd 중심) ■ 오픈소스 및 현대 Linux 구조 개념 • GNU/Linux / POSIX / API / ABI • SysV init → systemd로의 변화 • modern Linux 구성요소 개요 ○ systemd ○ journald ○ networkd / NetworkManager ○ firewalld ■ systemd 전체 흐름 이해 • PID 1 역할 • Unit 종류(service, socket, timer, target 등) • 의존성(After, Requires, Wants) ■ 실습: systemd 기본 조작 • systemctl start/stop/restart • enable/disable • isolate, default target • override 디렉토리 구조 ■ 시스템 운영 중급 명령어 • lsof • ss • strace • mpstat/sar 개요
2일	systemd + journald 심화 ■ systemd 서비스 및 부팅 구조 상세 • 부팅 단계 • initramfs와 systemd 역할 • 서비스 활성화 흐름 • unit 파일 구조 • ExecStart / ExecReload / Restart 정책 ■ 실습: 커스텀 systemd 서비스 작성

	• 단일 서비스 • 환경변수 분리 • 서비스 override 설정 • timer 연동 ■ journald 구조 완전 이해 • syslog와 journald의 차이 • persistent storage • BOOT ID / FIELD 기반 로그 필터링 • PRIORITY 기반 조회 • 특정 서비스별 로그 필터 ■ 실습: journalctl 활용 • 디버깅용 로그 조회 • 타임라인 기반 필터 • 커널 로그 조회 • 사용자 세션 기반 필터
3일	현대 스토리지 계층(Stratis/LVM) 집중 실습 ■ 현대 리눅스 스토리지 계층 구조 • Block Layer → DM(Device Mapper) → Filesystem 구조 • LVM 기본 구조(Logical Volume / PV / VG / LV) • snapshot / thin-provision 개념 • Stratis 등장 배경 ■ Stratis 이론 + 실습 • Stratis 풀 생성 • 파일시스템 생성 • snapshot 관리 • 모니터링 및 상태 확인 ■ LVM 실습 • PV/VG/LV 생성 및 확장 • LV snapshot • XFS 파일시스템 확장/복구 • RAID-LVM 개념 소개 ■ swap/zram 구조 이해 • swap vs zram • 메모리 압축 구조 • 현대 커널에서의 메모리 동작 방식
4일	네트워크·보안·세션 관리·트러블슈팅 종합 ■ 현대 Linux 네트워크 구조 • ip 명령어 구조 • address / route / neigh

	• ss를 통한 포트/소켓 분석 • nftables 기본 이해 • firewalld와 nftables의 관계 ■ 실습: 네트워크 구성 및 문제 해결 • 고정 IP 설정 • DNS/게이트웨이 세팅 • ping/ss/ip로 문제 분석 • Routing 문제 해결 • 서비스 포트 충돌 해결 ■ 세션 관리(Session Recording) • TLOG 개념 • session recording 구성 • sudo 기록과 연동 • 보안 감사(Audit) 연동 흐름 ■ 성능 및 장애 분석 • sar/mpstat/vmstat • journalctl + systemctl 조합 문제 해결 • CPU/메모리/IO 병목 분석 • 부팅 실패 문제 해결(단일 모드 포함) ■ 종합 랩: 표준 리눅스 서버 구성 완성 • systemd unit 작성 • journald 구조화 • Stratis 기반 스토리지 • 네트워크 구성 • session recording 적용 • 장애 시나리오 복구
--	--

엔터프라이즈 리눅스 컨버전스 & 마이그레이션(RHEL·SUSE·Debian 계열 비 교와 전환 실습)

교육명	엔터프라이즈 리눅스 컨버전스 & 마이그레이션(RHEL·SUSE·Debian 계열 비교와 전환 실습)
수정	2026-01-30
교육개요	본 과정은 RHEL 중심으로 구성되어 온 기존 엔터프라이즈 리눅스 환경이 변화하 는 상황에서, 기업이 선택할 수 있는 대안 리눅스 배포판을 기술적·운영적 관점에 서 분석하고 실제 전환 가능성을 검증하는 실무 중심 교육입니다. Red Hat의 SRPM 재배포 정책 변화 이후, Rocky Linux 및 AlmaLinux는 완전한 RHEL 호환성 대신 ABI/kABI 수준의 호환성 유지 전략으로 전환하였습니다. 이로 인해 기업은 단순한 “무료 RHEL 대체재”가 아닌, 운영 철학·지원 모델·자동화

	생태계까지 고려한 배포판 선택이 요구되고 있습니다. 본 과정에서는 다음 질문에 답합니다. • RHEL 호환 배포판은 실제 운영에서 어디까지 대체 가능한가? • SUSE는 왜 금융·제조·공공에서 선택되는가? • Debian 계열은 서버 운영에 적합한가? • 커널, systemd, 네트워크, 패키지 관리, 자동화 도구 차이는 마이그레이션 비용에 어떤 영향을 주는가? 모든 실습은 OpenStack 기반 랩 환경에서 진행되며, 단순 설치 실습이 아니라 "기존 RHEL 서버를 다른 배포판으로 옮긴다면 무엇이 깨지는가"를 직접 확인하도록 설계되어 있습니다.
목표	엔터프라이즈 리눅스 배포판의 기술적·정책적 차이 이해 RHEL → SUSE/Debian 계열 전환 시 호환성 리스크 분석 능력 확보 커널, systemd, 네트워크, 보안, 자동화 관점에서 마이그레이션 영향도 평가 조직 환경에 맞는 현실적인 배포판 선택 기준 수립
기간	총 4일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	RHEL 기반 시스템을 운영 중이며 대체 배포판을 검토 중인 인프라/플랫폼 엔지니어 Rocky/Alma/SUSE/Debian 간 차이를 기술적으로 비교하고 싶은 운영자 라이선스·서포트·자동화까지 고려한 중장기 OS 전략이 필요한 조직 클라우드/가상화/OpenStack/Kubernetes 환경에서 OS 표준화를 고민하는 담당자
1일	엔터프라이즈 리눅스 지형 변화와 배포판 비교 - RHEL 라이선스 정책 변화와 영향 분석 - ABI / kABI 개념과 실제 운영 영향 - SUSE / RHEL / Debian 계열 철학 비교 - 무인 설치 도구 비교 ■ Kickstart vs AutoYaST vs Preseed - 패키지 관리 구조 ■ DNF / Zypper / APT 내부 구조 비교 - 관리 도구 비교 ■ YaST2 vs Red Hat TUI 도구 - 파일 시스템 전략 ■ XFS / Btrfs / EXT4 실무 선택 기준 - LSB 호환성의 현재 의미
2일	커널·시스템·네트워크 차이로 보는 이식성 - 리눅스 표준 명령어의 미묘한 차이 - 바이너리 & 라이브러리 호환성 실습 - 커널 모듈 관리 ■ DKMS / kABI 안정성

	- 블록 디바이스 관리 ■ udev / multipath / device-mapper - systemd 단위 파일 비교 - 네트워크 구성 방식 ■ wicked vs NetworkManager ■ YaST 네트워크 설정 실습
3일	보안·자동화·운영 도구 관점의 전환 - 방화벽 체계 비교 ■ firewalld / nftables / iptables - 패키지 저장소 구성 전략 - 자동화 도구 비교 ■ Ansible / Salt / Terraform - 커널 라이브 패치 ■ Kgraft vs kpatch - Cockpit 기반 서버 관리 ■ SUSE vs RHEL 차이 ■ Debian에서 사용하기.
4일	마이그레이션 시나리오와 실전 판단 - 가상화 & 메모리 기술 비교 - 컨테이너 런타임 및 표준 OCI 환경 - SUSE 기반 Vanilla Kubernetes 설치 가이드 - 운영 자동화 설계 예제 - 장애/트러블 슈팅 체크리스트 - 미들웨어 소프트웨어 호환성

2. 고급 리눅스(4일)

운영자를 위한 Bash 자동화 도구 제작

교육명	운영자를 위한 Bash 자동화 도구 제작(kubectl 래퍼부터 대화형 CLI까지)
수정	2025-12-26
교육개요	본 과정은 리눅스 운영 환경에서 널리 사용되는 Bash 셸 스크립트를 현대적인 운영 자동화 도구로 활용하는 것을 목표로 합니다. 단순 반복 작업 수준의 스크립트를 넘어, 대화형 CLI 도구, 운영자 친화적 명령어 래퍼, 쿠버네티스 관리 자동화 스크립트를 직접 설계하고 구현합니다. 특히 kubectl 명령어를 단순 실행하는 수준이 아니라, • 사용자 입력을 처리하고 • 환경을 자동 판별하며 • 오류를 제어하고 • 운영 표준을 코드로 강제하는 "운영 도구로서의 셸 스크립트"를 만드는 실습 중심 과정입니다.

	모든 실습은 OpenStack 기반의 실제 리눅스 VM 및 쿠버네티스 클러스터 환경에서 진행됩니다
목표	Bash 셸 스크립트를 운영 자동화 도구 수준으로 작성할 수 있다 함수, 조건문, 배열, 환경 변수를 활용한 구조화된 스크립트 설계 능력 확보 사용자 입력 기반의 대화형 셸 CLI 도구 구현 kubectl 명령어를 래핑하여 운영 표준화된 쿠버네티스 관리 도구 제작 SSH 없이 노드 및 리소스를 관리하는 운영자 관점의 자동화 방식 이해 향후 Ansible, Go, Python 기반 자동화로 확장 가능한 셸 기반 설계 감각 습득
기간	3일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	최소 1년 이상 리눅스 시스템 운영 경험자 셸 스크립트는 작성해봤으나, 운영 도구로 확장해본 경험이 없는 엔지니어 쿠버네티스를 운영 또는 학습 중인 인프라/플랫폼 엔지니어 Ansible, DevOps 자동화로 넘어가기 전 기초 자동화 체력을 다지고 싶은 실무자
1일	리눅스 셸 서비스와 실행 환경 이해 - Bash 실행 흐름과 로그인 셸 vs 비로그인 셸 - 셸 스크립트 실행 방식 차이 (source/exec/subshell) 현대적 Bash 스크립트 작성 가이드 - 표준 셸 스크립트 스타일 가이드 - 가독성과 유지보수를 고려한 스크립트 구조 - strict mode(set -euo pipefail)의 의미와 적용 시점 변수 · 함수 · 조건문 심화 - 지역 변수와 전역 변수 설계 - 함수 리턴 처리 패턴 - [[]] vs [] vs (()) 차이와 사용 기준 - 중괄호, 대괄호 중복 사용 시 주의점 시스템 변수와 환경 자동 판별 - OS/Kernel/Architecture 판별 - PATH, HOME, USER, UID 활용 - 실행 환경에 따른 분기 처리
2일	kubectl 명령어 구조 이해 - kubectl 내부 동작 방식 - context/namespace 관리 원리 kubectl Bash 래퍼 설계 - kubectl 명령어를 감싸는 함수 구조 - 공통 옵션 자동 삽입 - 네임스페이스 고정 및 강제 정책 적용 대화형 셸 도구 구현 - 사용자 입력(read, select) 처리

	- 메뉴 기반 CLI 설계 - 오류 상황 및 예외 처리 운영 자동화 기능 구현 - 사용자 로그인 및 권한 확인 - 클러스터 정보 조회 및 컨텍스트 전환 - 이벤트 로그 조회 및 필터링
3일	노드 관리 자동화 - 노드 상태 조회 및 필터링 - 노드 라벨/테인트 관리 스크립트화 - sshless 운영을 위한 kubectl 기반 관리 방식 자원 관리 자동화 - POD 생성 및 삭제 자동화 - 서비스 생성 및 상태 확인 도구 구현 - 인그레스 생성 템플릿화 - YAML 자동 생성 쉘 스크립트 로드 밸런서 관리 - MetalLB 리소스 생성 자동화 - IP Pool 관리 스크립트 - 서비스 타입별 자동 분기 처리

표준 리눅스 클러스터링 시스템(Pacemaker)

교육명	표준 리눅스 클러스터링 시스템
수정	
교육개요	현 리눅스 시스템은 물리/가상적으로 여러 운영체제에 설치하여 사용한다. 이러한 이유로 특정 노드에 장애가 발생시, 다른 노드에서 중지된 서비스를 그때로 이전을 시켜야 한다. 현재 리눅스 시스템은 Pacemaker라는 표준적인 High Availability 시스템을 사용하고 있다. 어떠한 방식으로 설치 및 운영하는 기본적인 교육을 통해서 확인한다.
목표	리눅스 기반으로 H/A 시스템 구축 및 운영에 대해서 학습한다.
기간	총 5일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	최소 1년 이상의 리눅스 시스템 운영자 혹은 서비스 아키텍트 오픈소스 H/A 시스템이 구축 및 운영이 필요한 인프라 엔지니어 베어메탈 혹은 VM 기반으로 네이티브 H/A 시스템 구현이 필요한 인프라 엔지니어
1일	페이스메이커 소개 - 페이스메이커 구조 및 기능 설명 - Corosync 및 Agent 설명

	- LSB/OCF에이전트 설명 페이스메이커 랩 구성 - 하이퍼브이 기반으로 구성 - 랩 스토리지 생성 페이스메이커 설치 - 페이스 메이커 설치 - 기본적인 클러스터 구성 및 설치 - 노드 추가 및 삭제 방법 - pcs명령어 사용방법
2일	표준 서비스 기반으로 리소스 구성 - 기본 리소스 생성 및 관리 - 리소스 설정 및 구성 DRBD기반 스토리지 관리 - DRBD구성 - DRBD기반으로 스토리지 복제 및 공유 펜싱 장치 - 펜싱 장치 설명 - 펜싱 장치 생성 및 구성 페이스메이커 웹 관리자 - 웹 관리자 구성 및 접근 - 사용방법 페이스메이커 ACL/ALERT - ACL 설명 - ACL기반으로 사용자 접근 관리 - ALERT 설명 - ALERT기반으로 알람 설정
3일	페이스메이커 Booth - Booth설명 - 간단한 설치 및 구성 설명(실제로 설치하지 않음. 고급정에서 추후) 페이스메이커 기반의 DR(Disaster Recovery) 구성 - DR 서비스 설명 - 지원하는 DR 서비스 유형 - DR 서비스 구성 및 구현 QDEVICE - QDEVICE 설명 - 간단하게 설치 및 구현 리소스 생성 - 리소스 생성

	- 리소스 관리 및 펜싱 리소스 점수(Resource score/infinity) - Score, infinity 설명 - Location, co-location, resources, stickiness LVM2기반 스토리지 관리 - LVM2 리소스 설명 - LVM2 구성 및 랩 실험 NFS기반 스토리지 관리 - NFS 리소스 설명 - - NFS 구성 및 랩 실험
4일	아파치 서비스 - 설명 - 서비스 구성 및 실험 GFS2 스토리지 - 설명 - 서비스 구성 및 실험 톰캣 서비스 - 설명 - 서비스 구성 및 실험 PgSQL/Mariadb - 설명 - 서비스 구성 및 실험 TWO Node 구성 - 설명 - 액티브/스탠바이 구성 및 실험a - - 액티브/액티브 구성 및 실험

리눅스 트러블 슈팅

교육명	표준 리눅스 트러블 슈팅
수정	2025-12-26
교육개요	본 과정은 systemd 기반 리눅스 환경에서 실제 운영 중 발생하는 부팅 장애, 파일 시스템 손상, 블록 디바이스 오류, 네트워크 및 커널 문제를 원인 분석 → 복구 → 재발 방지 관점에서 다루는 실무 중심 트러블 슈팅 과정입니다. XFS, LVM, Btrfs, systemd, SELinux, cgroup 등 현대 리눅스 표준 컴포넌트에서 발생하는 장애 시나리오를 직접 재현하고, 로그·시스템 상태·커널 정보를 기반으로 문제를 진단하는 능력을 기릅니다.
목표	systemd 환경에서 발생하는 부팅·서비스·블록 디바이스 장애 분석 파일 시스템(XFS, LVM, 확장 FS) 및 스토리지 장애 원인 파악과 복구

	네트워크 및 커널 영역 장애 분석 시스템 콜 제한, SELinux, cgroup 관련 장애 이해 패키지 관리자(RPM/DNF) 및 시스템 무결성 문제 해결
기간	4일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	최소 1년 이상의 리눅스 시스템 운영자 혹은 서비스 아키텍트
교육대상	리눅스 운영 담당자 리눅스 사용 경험이 2년 이상 운영자 AI 및 클라우드 시스템 운영 담당자
1일	systemd에 통합된 시스템 영역 확인 - systemd 아키텍처 구조 분석 • unit 파일 구조(.service, .mount, .target) • 의존성(After, Wants, Requires) 분석 방법 - 서비스 기동 실패 시 systemctl 기반 원인 추적 램 디스크를 통한 운영체제 복구 및 파일 시스템 확인 - initramfs의 역할과 부팅 실패 시 동작 방식 - rescue/emergency 모드 진입 실습 - 루트 파일 시스템 마운트 실패 상황 재현 및 복구 grub2, bootrec를 통한 부트로더 재구성 및 복구 - GRUB2 구성 요소 이해 - 부트 메뉴 손상, 커널 파라미터 오류 상황 분석 - GRUB2 재설치 및 설정 복구 실습 - UEFI/BIOS 환경 차이점 이해 부트 업 장치 복구 및 영구적 마운트 장치 수정(/etc/fstab, .mount) - /etc/fstab 오류로 인한 부팅 실패 분석 - UUID/LABEL 기반 마운트 전략 - systemd .mount 유닛 기반 마운트 관리 - 디스크 교체 시 발생하는 마운트 장애 대응 부트 업 속도 확인 및 속도 개선 - 서비스별 부팅 시간 분석 - 불필요한 서비스 비활성화 전략 - 실제 운영 환경에서의 부팅 병목 사례 분석 파일 시스템 복구(xfs 및 LVM2) - XFS 파일 시스템 손상 분석 및 복구 - LVM 구조 이해 및 메타데이터 복구 - 블록 디바이스 오류 로그 해석 - 스토리지 장애 시 운영 판단 기준 블록장치 용량 확장(LVM2, Stratis)

	- LVM 기반 온라인 디스크 확장 - Stratis 개념 및 활용 시나리오 - 확장 시 파일 시스템별 주의사항 - 블록장치 장애 확인 및 처리 및 수정
2일	systemd-journald 통한 시스템 이벤트 분석 및 로그 분석 - journald 구조 및 로그 저장 방식 - 부팅 로그, 서비스 로그 분석 실습 - 시간 기준 / 서비스 기준 로그 추적 - 기존 rsyslog와의 차이점 이해 coredump 활성화 간단한 장애 발생 및 분석 - coredump 활성화 및 수집 구조 - 애플리케이션 크래시 상황 재현 - 기본적인 덤프 분석 흐름 이해 - 운영 환경에서 coredump 관리 전략 SELinux 컨텍스트 및 시스템 콜 장애처리 - SELinux 정책 구조 이해 - permissive / enforcing 차이 - AVC 로그 분석 및 원인 파악 - 임시 조치 vs 영구 조치 판단 기준 네트워크 설정 및 장애처리(NetworkManager/systemd-networkd) - NetworkManager 구조 및 동작 원리 - 인터페이스 다운, 라우팅 오류 분석 - VLAN, MTU 불일치로 인한 장애 사례 - nmcli 기반 네트워크 복구 실습 성능 향상 및 분석을 위한 모니터링 방법 - CPU, Memory, Disk, Network 병목 분석 방법 - 기본 모니터링 도구 활용 - 장애 전조 패턴 식별 방법 LLM을 활용한 로그 요약 및 장애 원인 추론 - systemd-journald 로그를 사람이 보기 힘든 이유 - LLM을 활용한 로그 요약/장애 패턴 분류/의심 지점 후보 추출
3일	커널 모듈 및 파라미터 확인 및 수정 - 커널 모듈 로딩/언로딩 장애 분석 - sysctl 파라미터 변경 실습 - 커널 파라미터 오설정으로 인한 장애 사례 메모리 페이징 및 스왑 장애처리 그리고 확장 - 메모리 사용 구조 이해 - 스왑 설정 오류로 인한 성능 저하 사례

	- OOM 발생 전후 로그 분석 - 스왑 확장 및 조정 실습 리눅스 시스템 사용자 세션 관리 및 분석(사용자 콘솔 로깅 포함) - 사용자 로그인 세션 구조 이해 - SSH / 콘솔 접근 장애 분석 - 다중 로그인 세션 관리 - 사용자 행위 기반 로그 추적 보고 및 분석을 위한 시스템 기록 수집 및 분석 - 장애 보고를 위한 로그 수집 전략 - 장애 타임라인 재구성 방법 - 운영 보고서 작성을 위한 데이터 정리 방식
4일	QEMU/KVM, Libvirt 장애 분석 및 처리를 위한 방법 제안 - 가상머신 기동 실패 원인 분석 - libvirtd 로그 기반 문제 추적 - 스토리지·네트워크 연계 장애 사례 - OpenStack 환경과의 연관성 이해 컨테이너 장애 분석 및 처리를 위한 방법 제안 - 컨테이너 기동 실패 원인 분류 - 호스트 리눅스 관점에서의 컨테이너 장애 분석 - 네임스페이스, cgroup 기반 리소스 제한 이해 메모리 관리 OOM, CGROUP 확인 및 장애처리 - cgroup 구조 및 리소스 제한 방식 - OOM Killer 동작 원리 - 컨테이너 환경에서의 OOM 분석 RPM 및 DNF3 장애 처리 - 패키지 의존성 오류 분석 - 깨진 패키지 DB 복구 - 업데이트 중 장애 발생 시 대응 전략 확장 퍼미션(attr) 통한 장애 처리 및 확인 - ACL, xattr 개념 이해 - 퍼미션 정상인데 접근 불가능한 사례 분석 - 보안·백업·컨테이너 환경에서의 영향

대규모 리눅스 노드 설치 및 구성 자동화

교육명	대규모 리눅스 노드 설치 및 구성 자동화
수정	2025-12-26
교육개요	온프레미스 및 퍼블릭 클라우드에 많은 컴퓨팅 자원들이 생성 및 구성이 된다. 특히, 많은 회사들이 온프레미스 인프라로 다시 롤백 혹은 전환하면서 대용량 노

	드 관리에 대해서 많은 어려움이 있다. 이를 해결하기 위해서 오픈소스에서 많이 사용하는 Foreman(Satellite)기반으로 큰 규모의 컴퓨팅 노드 구성 및 관리를 학습하도록 한다.
목표	여러 물리적 서버 및 가상서버 관리 방법에 대해서 학습 프로비저닝이 완료된 서버에서 앤서블과 같은 도구로 구성하는 방법학습 foreman기반으로 여러 리눅스 배포판 구성 및 업데이트에 대해서 학습
기간	4일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	대형 인프라를 관리 및 운영하는 시스템 관리자 인프라 설치 및 자동화가 필요한 시스템 관리자 프로비저닝 및 디플로이먼트 통합이 필요한 시스템 관리자
1일	The Foreman 설명 - Foreman 아키텍처 - Foreman 주요 기능 - 기존 관리 방식과 Foreman의 차이점 및 장점 The Foreman 설치 - 설치 전 요구사항 - 통합 부분 ■ puppet, ansible, docker(podman) 통합 ■ 네트워크 및 호스트 The Foreman 설치 - 레드햇 계열 - 데비안 계열 - 수세 계열 - 설치 및 기본설정 - Foreman 웹 인터페이스 사용 The foreman 기본 구성 - 호스트 등록 - - O/S 템플릿 구성
2일	호스트 관리 - 호스트 생성 및 등록 - 호스트 그룹 생성 및 등록 - OS 템플릿 기반으로 프로비저닝 구성 프로비저닝 환경 설정 및 구성 - 환경 분리 구성하기(dev, test, product) - 환경별 호스트 그룹 및 설정 - 프로비저닝 이후 puppet/ansible/docker(podman) 자동화 도구와 통합 호스트 배포 및 관리

	- OS배포 - 호스트 상태 관리 및 모니터링 - 클라우드 시스템에서 프로비저닝 자동화 Puppet/ansible 통합 - 통합 시나리오 설명 - - 앤서블 기반으로 설치 후 디플로이먼트 구성
3일	Foreman 확장 기능 - 확장 모듈 - 템플릿 구성 및 관리 - RPM 저장소 관리 및 구성 - 사용자 관리

리눅스 하드닝(보안)

교육명	표준 기반 리눅스 하드닝 및 침해 대응 실무(NIST · ISMS-P · 자동화 · CVE 실습)
수정	2025-12-26
교육개요	본 교육은 NIST SP 800-53 및 KISA ISMS-P 보안 통제 기준을 기반으로, 실무 Linux 시스템에 적용 가능한 보안 하드닝, 접근 제어, 자동화 구성, 무결성 검증 및 CVE 기반 침해 대응을 단계적으로 학습합니다. 단순 설정 나열이 아닌, 운영 환경에서 실제로 발생하는 보안 사고 시나리오를 중심으로 보안성 강화와 운영 안정성을 동시에 만족하는 보안 운영 방법을 실습 중심으로 다룹니다. 본 과정은 인프라 블루팀 관점의 기본 운영을 중심으로 진행됩니다.
목표	NIST 및 KISA 기준 보안 정책을 이해하고 상호 비교할 수 있다. Linux 시스템 접근 제어 및 서비스 하드닝을 수행할 수 있다. Ansible을 활용하여 보안 설정을 자동화할 수 있다. 실제 CVE 기반 침해 시나리오를 분석하고 대응할 수 있다.
기간	4일 (총 28시간)
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	Linux 시스템 관리자 보안 담당자 DevSecOps 입문자
1일	보안 하드닝 개요 및 접근제어 - NIST와 KISA 보안 정책 구조 비교 - Linux 보안 사고 사례 분석 - 계정 관리 및 패스워드 정책 설정 - su/sudo 제한, PAM 설정 이해

	[실습] - /etc/passwd, /etc/shadow 보호 - login.defs, pam_limits.conf 설정 - 사용자 권한 감사
2일	네트워크 보안&Ansible 운영 표준 강제화(자동화) - SSH 하드닝, 포트 변경, 키 인증 - iptables/nftables/firewalld 설정 - logrotate, journald 로그 정책 - Ansible로 하드닝 정책 자동화 [실습] - Ansible로 SSH 운영 표준 강제화 - firewall-cmd 구성 - logrotate 정책 작성 및 테스트
3일	파일 시스템 보호 및 불변 시스템 이해 - chattr, lsattr를 통한 파일 보호 - /usr, /boot 읽기 전용 마운트 실습 - MicroOS, CoreOS vs 일반 배포판 비교 - SELinux/AppArmor 보안 정책 이해 ■ 앞으로의 커널 및 프로세서 보안미래 [실습] - chattr +i 적용 실습 - SELinux 설정 및 로그 확인
4일	CVE 기반 모의 해킹 & 무결성 검증 - CVE-2021-4034(polkit) 권한 상승 실습 - auditd, aide, rpm -Va 무결성 검증 이해 - Ansible로 패치 적용 자동화 - 침해 흔적 포렌식 및 대응 전략 [실습] - polkit CVE 실습 - 무결성 검사 및 포렌식 대응 시나리오

컨테이너 & 쿠버네티스

1. 컨테이너 어드민

표준 포드만 컨테이너 어드민

교육명	표준 포드만 컨테이너 어드민
수정	2025-12-26

교육개요	본 교육은 리눅스 및 오픈소스 생태계에서 표준 컨테이너 기술로 자리 잡고 있는 Podman과 Kubernetes에서 사용되는 CRI-O 런타임의 구조와 운영 방법을 학습하는 과정입니다. 기존 Docker 중심의 컨테이너 운영 방식에서 벗어나, OCI(Open Container Initiative) 및 CRI(Container Runtime Interface) 표준을 기반으로 한 컨테이너 아키텍처를 이해하고, Podman을 활용한 Pod 및 Container 생성·관리, API 기반 제어, systemd 연동 운영까지 실습 중심으로 다룹니다. 또한 Docker, Containerd, Podman, CRI-O 간의 역할과 차이를 명확히 비교하고, Kubernetes와 컨테이너 런타임의 관계를 구조적으로 이해할 수 있도록 구성되어 있습니다.
목표	Docker/Containerd와 Podman/CRI-O 기반 컨테이너 구조의 차이를 이해한다 Podman을 활용하여 Pod 및 Container를 생성·관리할 수 있다 Podman API 서버를 활용한 원격 관리 방식을 이해한다 표준 컨테이너(OCI/CRI) 기반 운영 방식과 Kubernetes 연계 구조를 이해한다
기간	총 4일, 3일로 조정가능 과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	최소 1년 이상 컨테이너 운영 경험자 도커에서 표준 컨테이너로 이전하려는 개발자 혹은 인프라 엔지니어
1일	표준 컨테이너 개념과 Podman 이해 - 표준 컨테이너란 무엇인가 ■ 기존 컨테이너 기술과의 차이점 ■ Podman, CRI-O, Docker, Containerd 비교 ■ 가상화와 컨테이너의 차이점 - Kubernetes와 컨테이너 런타임의 관계 ■ Kubernetes에서 지원하는 컨테이너 런타임 ■ Podman은 왜 Kubernetes 런타임으로 사용되지 않는가 ■ Kubernetes에서 사용 가능한 표준 컨테이너 도구 ■ 컨테이너 표준 구성 요소: containers, CNI, CSI - Podman 설치 ■ Podman 패키지 구조 이해 ■ Podman 설치 ■ Podman 서비스 및 환경 확인 - Podman 기본 명령어 ■ 컨테이너 관리 명령어 ■ Pod 관리 명령어 ■ 네트워크 명령어

	■ 스토리지 명령어
2일	표준 컨테이너 내부 구조와 Pod 기반 운영 - 기본 컨테이너 기술 ■ cgroup, namespace ■ unshare ■ container monitor(conmon), runc / crun ■ Pod 개념과 pause 컨테이너 ■ Kernel, init, 그리고 Pod의 관계 ■ NSPID, SUBID, UID/GID (rootless vs rootful) - 특수 권한 컨테이너 생성 - Pod 기반 컨테이너 생성 및 관리 ■ Pod 생성 및 관리 ■ Pod와 Container 연결 구조 ■ Pod 내부 컨테이너와 애플리케이션 관계 - 네임스페이스 기반 네트워크 구조 확인 - 표준 컨테이너 디렉터리 구조 ■ /run/containers ■ /run/pod ■ /var/lib/containers - OverlayFS 동작 방식 이해 - 표준 컨테이너 이미지 생성 - podman build - buildah - 이미지 검색 및 복사 - /etc/containers 설정 - skopeo
3/4일	다중 Podman Node 관리 및 운영 - Podman API 서버 - 다중 서버 접근/관리 및 서비스 배포 포드만에서 쿠버네티스로 서비스 테스트 - 쿠버네티스 서비스로 전환방법 컨테이너 기반으로 systemd서비스 구성 - 사용자 기반 systemd container 서비스 - 관리자 기반(root) systemd container 서비스

2. 컨테이너 엔지니어

표준 쿠버네티스(컨테이너/가상화)

교육명	표준 쿠버네티스(컨테이너/가상화)
수정	2025년 12월 02일

교육개요	본 과정은 클라우드 네이티브 인프라의 핵심 기술인 Kubernetes를 설치 → 구조 이해 → 스토리지 구성 → 네트워크/Ingress 구성 → 운영/업그레이드 → 모니터링/백업 순으로 체계적으로 익히는 실무 중심 과정입니다. 쿠버네티스의 핵심 구성요소, Control-Plane 구조, Overlay 네트워크, 스토리지 (CSI/NFS) 구성, 모니터링, ETCD 백업, 인그레스 및 로드밸런서 구성, 장애 처리와 운영 자동화까지 현장에서 필요한 운영자 기술을 집중 실습합니다. 교육은 OpenStack 기반 랩 환경에서 진행되며 실제 VM 기반의 온프레미스 구조에서 클러스터 구성과 운영을 경험할 수 있습니다.
목표	Kubernetes Control-Plane 및 Node 구성 원리 이해 kubeadm 기반 설치 구조 및 운영 패턴 학습 Pod/Deployment/Service 등 주요 리소스 실습 CSI 기반 스토리지 구성, NFS 기반 StorageClass 구축 Ingress/LoadBalancer/NetworkPolicy 등 네트워크 구조 학습 ETCD 백업/복원 및 클러스터 유지보수 역량 확보 Prometheus + Grafana 기반 모니터링 구성 온프레미스 환경에서 안정적인 운영을 위한 Troubleshooting 능력 습득
기간	총 5일
랩	OpenStack 기반 랩
교육대상	컨테이너 기술을 기반으로 인프라 운영을 하려는 엔지니어 온프레미스 또는 프라이빗 클라우드 환경에서 K8s 운영 예정인 관리자 DevOps 이전 단계에서 클러스터 운영 중심 역량을 키우고 싶은 실무자 OpenStack 기반 K8s 랩을 직접 운영해보고 싶은 교육 담당자
1일	Kubernetes 소개 및 아키텍처 - Cloud Native & OCI 개념 - Kubernetes 구성요소(Control-Plane/Node) - etcd/kube-apiserver/scheduler/controller-manager - containerd와 CRI 구조 - 가상화 vs 컨테이너 vs K8s 차이 설치 준비 - kubeadm 설치 개념 - 네트워크 플러그인 개요 (Flannel, Calico 등) - 노드 요구사항 - Cgroup/방화벽/커널 옵션 확인 실습: 클러스터 설치 - Control-Plane 설치 - Worker Node 조인 - Flannel 네트워크 구성 - kubectl 설정 및 자동완성 기본 리소스 실습

	- Namespace - Pod / Multi-container Pod - Deployment - Service(ClusterIP / NodePort)
2일	애플리케이션 관리 & 운영 기본 기능 kubectl 운영 명령 - get/describe/log/exec - edit/apply/delete - rollout / history - label/annotate/selectors - port-forward Pod 스케줄링 및 상태 분석 - readiness/liveness probe - Pod 상태 확인 - scheduling 흐름 이해 - NodeSelector / Affinity 기본 - POD기반 가상머신 생성 및 관리 가상머신 이미지 관리 - 가상머신 이미지 빌드 - 이미지 패키징 및 가져오기 - 컨테이너 이미지 레지스트리 Config/Secret 관리 - ConfigMap - Secret - 환경변수/Volume 연동 Deployment 운영 패턴 - 롤링 업데이트 - 롤백 - ReplicaSet/HPA 기본
3일	스토리지(온프레미스 환경 핵심) 온프레미스 Kubernetes 스토리지 구조 - PV / PVC / StorageClass - CSI 구조 개념 - On-prem Storage 모델(NFS/iSCSI/Ceph 개요) 실습: NFS 기반 스토리지 구성 - 외부 NFS 서버 구성 - CSI-Driver-NFS 설치 - StorageClass 생성 - PVC 바인딩 테스트

	- Pod에 Volume 마운트 실습 StatefulSet & 스토리지 활용 - StatefulSet 동작 방식 - Headless Service - 볼륨 재사용 패턴 - DB 클래스 애플리케이션 실습(MySQL/Redis)
4일	네트워크 & Ingress/LoadBalancer Kubernetes 네트워크 구조 이해 - Pod → Node → Cluster → External - CNI 구조 설명 - kube-proxy 동작(IPVS 중심) - NodePort의 한계 NetworkPolicy - Ingress/Egress 구조 - Deny-all → allow 모델 실습 - 네임스페이스 기반 통제 Ingress & LoadBalancer - Ingress 기본 구조 - IngressController(NGINX) 설치 - Path 기반 라우팅 - TCP/UDP 패스스루 - L4/L7 차이 이해 - API Gateway, 그리고 Ingress의 미래 왜 cert-manager가 필요한가 - 수동 TLS 관리의 한계 - 인증서 만료 사고 사례 - cert-manager 아키텍처 ■ Issuer / ClusterIssuer ■ Certificate / Challenge - 인증서 유형 ■ Self-signed ■ Internal CA ■ (옵션) ACME 개념 설명 ■ 실습: 도메인 기반 라우팅 - 테스트 서비스 배포 - Ingress로 여러 서비스 노출 - 인증서 적용(Optional) - cert-manager 설치 - Self-signed Issuer 생성

	- 테스트 도메인 인증서 발급 - Ingress에 TLS 적용 - 인증서 자동 갱신 동작 확인
5일	운영·백업·모니터링 & Troubleshooting ■ 클러스터 운영 자동화 - Drain / Cordon / Uncordon - Node 추가/삭제 - Cluster Upgrade 전략 ■ ETCD 백업 및 복원 - snapshot 저장 - 복원 절차 실습 - HA 구조 고려사항 ■ 로그 및 문제 해결 - kubectl debug - events 분석 - Pod crashloop 문제 파악 - imagePullBackOff 해결 패턴 ■ 모니터링(Prometheus + Grafana) - kube-state-metrics - node-exporter - Prometheus 구성 - Grafana 대시보드 연동 ■ 종합 랩 (종합 인프라 구성) • 스토리지 + 네트워크 + 인그레스 + 앱 배포 + 모니터링 • 실제 장애 상황 제공 후 복구 실습

수세 컨테이너/가상화 클라우드 어드민

교육명	수세 컨테이너/가상화 클라우드 어드민
수정	2025-12-23
교육개요	수세에 제공하는 REK2/RANCHER 그리고 SUSE VIRT(Harvester)를 통해서 컨테이너/가상화 통합 플랫폼 운영에 대해서 학습한다. 현재, 대다수 쿠버네티스 플랫폼은 가상화를 지원하고 있으며, 때로는 이를 독립적으로 사용하기도 한다. 어떠한 방식으로 통합 운영이 가능한지, 스토리지 구성에 대해서 교육을 통해서 확인한다.
목표	RKE2 및 기존 쿠버네티스의 차이점에 대해서 학습한다. 랜처 기반으로 하나 이상의 클러스터를 관리 및 운영하는 방법에 대해서 학습한다. 쿠버네티스 가상화를 통한 시스템 통합 운영에 대해서 학습한다.

	고가용성 스토리지를 통한 볼륨 및 디스크 운영에 대해서 학습한다.
기간	총 4일, 3일로 조정가능 과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	바닐라 쿠버네티스가 버전보다 안정적으로 운영이 가능한 쿠버네티스 클러스터 시스템을 고려하고 있는 인프라 엔지니어 다중 쿠버네티스 클러스터 운영이 필요한 인프라 엔지니어 가상머신 및 컨테이너 시스템 통합 운영이 필요한 인프라 엔지니어 랜처를 통한 통합 및 다중 클러스터 운영이 필요한 인프라 엔지니어
1일	랩 및 솔루션 소개 - 쿠버네티스와 랜처와 관계 - K8S와 K3S의 차이점 - 업/다운 스트림 버전 차이점 - 수세와 랜처의 관계 - 랜처 아키텍처 랜처 설치 - 운영체제 설정 - 랜처 및 쿠버네티스 설치 - 네트워크 구성 및 확인 랜처 관리자 도구 설치 - 관리자 도구 설치 - - 랜처 대시 보드 설명
2일	랜처 활용(기본 자원 관리) - namespace 및 POD 개념 - POD 생성 및 관리 - 자원 생성방법(apply, create) - POD와 Container관계 - POD 배포를 관리하는 Deployment, 그리고 RelicaSet - POD 구성을 위한 기본 활용 자원들 - 다중컨테이너 구성 - 서비스 생성 및 노출 - 레이블 및 선택조건(labels, match) - debug 및 log 확인 랜처 스토리지 구현 - CSI와 쿠버네티스 스토리지 관계 - NFS기반으로 구성 및 연결 - PV/PVC그리고 스토리지 클래스 간단한 설명 - PV/PVC그리고 스토리지 클래스 간단한 설명 쿠버네티스 패키지

	- Helm 패키지 - HelmChart 기반으로 프로그램 설치 - Rancher에서 Operator 사용하기
3/4/5일	고급 자원 관리 - 쿠버네티스 고급 자원관리 랜처에서 CI/CD구성 및 구현 - 아키텍처 설명 - GIT 서버 설치 및 구성하기 - 레지스트리 저장소 설치 및 구성하기 - 랜처 기본 기능 fleet 설명 - 표준 CNCF Tekton/ArgoCD 설명 - 위 도구를 통한 CI/CD 인터페이스 구성 - YAML 기반으로 테스크 및 파이프라인 구성 랜처 가상화 - 수세 가상화(harvester) 설명 - harvester 설치 방법 - 독립적으로 수세 가상화/통합으로 수세 가상화 활용 - 수세 가상화 기반으로 인스턴스 생성 - 스토리지 할당 및 가상머신 기능 활용

Okd 컨테이너 오케스트레이션(OpenShift 오픈소스 버전)

교육명	Okd 컨테이너 오케스트레이션
수정	
교육개요	레드햇 오픈시프트 커뮤니티 버전 OKD기반으로, 오픈시프트 설치 및 구성에 대해서 확인한다. 설치 후, okd기반으로 서비스 배포 및 서비스 구성을 하면서, 쿠버네티스와 어떠한 차이점이 있는지 확인한다. okd는 커뮤니티 버전이기 때문에, 기술 지원은 받을 수 없지만, 다운 스트림 버전 OpenShift와 기능적으로 동일하기 때문에 Bad Test용도 혹은 중소규모로 운영 시, 적절한 제품이다.
목표	okd기반으로 오픈시프트 기능 확인 및 설치 클러스터 운영 및 CI/CD구성
기간	총 4일, 3일로 조정가능 과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	리눅스 명령어 및 사용 경험자 쿠버네티스 명령어 및 사용 경험자
1일	OKD 소개 - OKD와 OpenShift의 차이점 - 쿠버네티스와 OKD의 차이점 및 역할 - 표준 도구

	클러스터 설치 - OKD설치 방법(IPI/UIP방식) - 클러스터 아키텍처(컨트롤러 및 컴퓨트) - 설치 전 L4 및 DNS 준비 - OKD 설치(프로덕트 모델 기반) - - 기본 명령어 및 서비스 살펴보기
2일	자원 관리 - 기존 쿠버네티스 자원 명령어 확인 ■ kubectl, oc 명령어 ■ 기존 쿠버네티스 자원 생성 및 관리 ■ okd project/kubernetes namespace POD 및 Deployment 그리고 DeploymentConfig의 차이점 - 애플리케이션 생성 - 애플리케이션 배포 및 관리 - 배포전략 네트워크 - okd SDN 네트워크 ■ Kubernetes SDN, OVN - okd router ■ router기반으로 서비스 라우팅 구성 - okd ingress ■ okd에서 ingress 서비스 사용하기 ■ ingress와 router의 차이점 okd 스토리지 - okd에서 제공하는 스토리지 ■ 스토리지 백엔드 추가 ■ PV/PVC ■ StorageClass okd 사용자 - okd 사용자 관리 ■ 사용자 백엔드 구성 ■ 사용자 추가 - - rbac기반으로 자원 접근 제한
3일	빌드 설정 - 이미지 빌드를 위한 S2I - 컨테이너 이미지 빌드 - 빌드 및 애플리케이션 배포 - 템플릿 기반으로 이미지 배포

	okd 오퍼레이터 - okd 오퍼레이터 설명 - 오퍼레이터 기반으로 클러스터 서비스 확장 okd helm - okd와 helm 활용하기 - helm chart 설치 및 관리 okd package - kustomized기반으로 패키징 - helm chart기반으로 애플리케이션 패키징 - 사용자 operator 패키징
4일	모니터링 및 디버깅 - Prometheus, Grafana, EKF 기반으로 리소스 모니터링 - POD 및 노드 자원 상태 확인 - 노드 접근 및 디버깅 CI/CD 구성 - okd기반에서 CI/CD 구성 - okd build 사용하기 - Tekton기반으로 CI/CD - ArgoCD기반으로 자원 배포 및 확인하기 - GIT 및 이미지 서버

클라우드 네이티브를 위한 MSA 구현

교육명	클라우드 네이티브를 위한 MSA 구현
수정	2025-12-23
교육개요	본 과정은 쿠버네티스 기반 클라우드 네이티브 환경에서 MSA(Microservices Architecture)를 설계하고 운영하는 방법을 학습하는 실습 중심 과정입니다. 수강생은 CNCF 표준 도구 체계를 기반으로 • CI/CD 파이프라인을 구성하고 • 레거시 애플리케이션을 컨테이너 및 MSA 구조로 전환하며 • 서비스 메시(Istio)와 서버리스 플랫폼(Knative)을 활용해 자동 확장·이벤트 기반 서비스 운영 방식을 직접 경험합니다. 본 과정은 애플리케이션 개발보다는 플랫폼 구성, 배포 흐름, 운영 관점의 MSA 이해에 초점을 맞추며 개발자는 Knative를 통해 애플리케이션을 "어떻게 배포하고 운영에 올리는지"를 체험합니다.
목표	쿠버네티스 기반 클라우드 네이티브 플랫폼 구성 흐름 이해 MSA 기반 애플리케이션 구조와 배포 방식 이해 CI/CD를 통한 컨테이너 빌드 및 GitOps 기반 배포 경험 Istio를 활용한 서비스 트래픽 관리 및 가시성 확보 Knative 기반 이벤트 드리븐 서비스 배포 및 자동 확장 구현 인프라 자원 생성 없이 서비스 중심으로 운영하는 클라우드 네이티브 방식 습득
기간	총 4일, 3일로 조정가능 과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	리눅스 서버 운영 3년 이상 쿠버네티스 기본 개념 및 kubectl 사용 경험 있음 컨테이너 이미지 빌드·배포 경험 있음 개발자가 작성한 애플리케이션을 "<u>어떻게 운영 환경에 올릴지</u>" 고민해본 사람
1일	클라우드 네이티브 플랫폼 기초 - 쿠버네티스 아키텍처 및 클러스터 구성 이해 - (간소화된) 쿠버네티스 환경 준비 - CI/CD 개념과 CNCF 도구 체계 이해 - ArgoCD(GitOps) 개념 및 배포 흐름 - Tekton 기반 컨테이너 빌드 파이프라인 이해 - buildah/podman을 이용한 이미지 빌드 및 테스트
2일	레거시 → MSA 전환 - 레거시 애플리케이션의 컨테이너화 전략 - 레거시 → 쿠버네티스 마이그레이션 패턴 - MSA 구성을 위한 서비스 분리 개념 - 간단한 MSA 애플리케이션 구성 및 배포 - CI/CD 파이프라인과 연계한 배포 자동화

3일	Istio 기반 서비스 메시 - Istio 아키텍처 및 역할 - 서비스 메시가 필요한 이유 - Istio 설치 및 기본 구성 - 트래픽 제어, 관측(Tracing, Metrics) - 서비스 스케일링과 장애 대응 개념
4일	Knative 기반 서버리스 MSA - Knative 개념 및 Kubernetes와의 관계 - Knative Serving/Eventing 구조 - Knative 서비스 배포 - 이벤트 기반 서비스 구성 - 자동 스케일링(Scale to Zero 포함) - 클라우드 네이티브 MSA 운영 시나리오 정리

가상화 인프라

1. 가상화의 바닥을 이해하는 운영자 과정

교육명	가상화의 바닥을 이해하는 운영자 과정
수정	2025-12-26
교육개요	본 과정은 리눅스 환경에서 표준적으로 사용되는 QEMU/KVM 기반 가상화 기술을 Libvirt(libvirtd)를 중심으로 이해하고 운영하는 실무 과정입니다. Libvirt의 구조와 동작 방식을 기반으로 가상머신 생성, 네트워크·스토리지 구성, 디스크 이미지 관리, 마이그레이션, 성능 튜닝 및 트러블슈팅까지 실제 운영 환경에서 필요한 핵심 내용을 단계적으로 학습합니다. 또한 Libvirt 기반 가상화가 OpenStack, Kubernetes, Harvester 등 현대적인 클라우드 및 컨테이너 플랫폼의 하부 인프라로 어떻게 활용되는지를 구조적으로 이해하고, 가상머신 위에서 컨테이너(Podman/Docker)를 운영하는 실무 시나리오를 통해 VM-컨테이너 혼합 인프라 환경에 대한 운영 관점과 장애 분석 역량을 함께 강화합니다. 본 과정을 통해 수강생은 단순한 가상머신 사용을 넘어, 가상화 계층의 근본 원리 이해를 바탕으로 클라우드 및 컨테이너 환경에서 발생하는 문제를 보다 깊이 있게 분석하고 대응할 수 있는 운영 능력을 갖추게 됩니다.
목표	Libvirtd명령어 및 기능에 대해서 이해한다. 동작 방법 및 기존 오픈스택 및 쿠버네티스와 어떠한 관계가 있는지 확인한다. 표준 도구를 통한 이미지 관리 및 배포 방법에 대해서 학습한다
기간	총 4일, 3일로 조정가능 과정
랩	오픈스택 기반 랩
교육대상	최소 1년 이상의 리눅스 운영 경험이 있는 사용자 VMware혹은 Xen 가상화에서 QEMU/KVM기반의 가상화로 기술 전환이 필요한

	가상화 인프라 엔지니어
1일	랩 구성 및 설명 QEMU/KVM 설명 - 동작 방식 및 QEMU/KVM관계 - 설치 및 서비스 확인 - KVM의 역할 - QEMU가 다루는 자원 Libvirt 설명 - Libvirt와 하이퍼바이저 차이점 - Libvirt 서비스/설정 및 디렉터리 - QEMU/KVM과 관계 virsh명령어 학습 - 자주 사용하는 명령어 활용 - virsh명령어 기반으로 XML파일 수정 - 자원 조회 및 활용 저장소 위치 구성 - 기본 저장소 위치 - 저장소 추가 및 드라이버 네트워크 설정 구성 - Linux Bridge Network - OpenvSwitch - VLAN 가상머신 디스플레이 - VNC - - Spice
2일	가상머신 관리 - 가상머신 생성 - 가상머신 마이그레이션 - 가상머신 디스크 이미지 ■ 스냅샷 생성 및 관리 ■ 이미지 편집 및 수정 ■ 가상머신 템플릿 이미지 ■ 이미지 저장소 구성 - 가상머신 자동 설정 ■ cloud-init 설명 ■ cloud-init 스크립트 및 명령어 마이그레이션 - 다중 노드에서 Libvirt 기반으로 마이그레이션 구현

3일	반가상화 드라이버 구성 및 사용 - 네트워크 - 스토리지 Guest Tools 설명 - 가상 자원 생성을 위한 QEMU - 디스크 관리를 위한 도구 - 이미지 배포 및 배포서버 - 자동 이미지 빌드 시스템 API기반으로 Libvirtd서버 관리 및 서비스 배포 - 다중 노드 기반으로 가상머신 구성 성능 개선을 위한 설정 및 튜닝 - 네트워크 장치 - 블록 디스크 장치 - 반가상화 장치 - NUMA - KVM KSM - De-Duplicate Block Service (VDO) libvirt 및 가상머신 트러블 슈팅
----	---

2. KubeVirt 기반 Kubernetes 네이티브 가상머신 운영

교육명	KubeVirt 기반 Kubernetes 네이티브 가상머신 운영
수정	2025-12-26
교육개요	컨테이너 기반 PaaS가 애플리케이션 배포의 중심이 되었지만, 운영체제 단위 격리, 레거시 워크로드, 상태 기반 서비스 영역에서는 여전히 가상머신 기반 아키텍처가 필요합니다. 본 과정은 이러한 요구를 Kubernetes 네이티브 방식으로 해결하기 위해 KubeVirt를 활용한 가상머신 통합 운영 모델을 실습 중심으로 학습합니다. 수강생은 Kubernetes 리소스로 가상머신을 생성·운영하고, 기존 OpenStack/VMware 환경과의 차이점 및 활용 시나리오를 실무 관점에서 이해하게 됩니다.
목표	Kubernetes 기반 가상화(KubeVirt)의 구조와 동작 원리 이해 Kubernetes 리소스로 가상머신을 생성·운영·자동화하는 방법 습득 기존 가상화(OpenStack/VMware)와 KubeVirt의 활용 시나리오 비교 이해
기간	총 3일
랩	오픈스택 기반 랩
교육대상	Kubernetes 기본 오브젝트(Pod, Deployment, Service, PV/PVC)를 이해하고 있는 엔지니어 IaaS(OpenStack, VMware 등) 또는 PaaS 환경 운영 경험이 있는 인프라/플랫폼 엔

	지니어 컨테이너와 가상머신을 동시에 운영해야 하는 환경을 고민하는 실무자
1일	Kubernetes에서 VM이 동작하는 방식 이해하기 - KubeVirt란 무엇인가? - Kubernetes와 KubeVirt의 역할 분리 - KubeVirt 아키텍처 핵심 구성요소 - 컨테이너와 가상머신 통합 모델 이해 설치 & 기본 운영 - KubeVirt 설치 아키텍처 - 사전 요구사항 및 노드 구성 - Helm 기반 KubeVirt 설치 - kubect/virtctl 기본 사용법 - VM 리소스(YAML) 구조 이해
2일	KubeVirt 가상머신 생성·운영 실습 - 가상머신 라이프사이클 ■ VirtualMachine/VirtualMachineInstance 이해 ■ YAML 기반 VM 정의 및 배포 ■ 디스크 이미지 관리 (PVC/DataVolume) 네트워크 - VM 네트워크 모델 이해 - VM ↔ Kubernetes 네트워크 통신 - VM 인터페이스 설정 실습 운영 관리 - VM 상태 관리 및 로그 확인 - 자원(CPU/MEM/DISK) 관리 - 스냅샷/복구/롤백 VM과 컨테이너의 관계 - VM 위 컨테이너 실행 구조 - Bare-metal vs VM 컨테이너 비교 - "컨테이너 실행용 VM" 설계 전략
3일	고급 기능 - VM 스케줄링과 노드 분산 - GPU/디바이스 패스스루 - VM 템플릿 전략 보안 - Kubernetes RBAC 기반 VM 제어 - 네트워크 접근 제어 - SELinux/AppArmor 활용 및 사용

	성능 최적화 - CPU Pinning/NUMA - Virtio 성능 튜닝 - 스토리지 캐시 전략 자동화 & 실무 패턴 - cloud-init 기반 VM 자동 구성 - VM 생성 시 컨테이너 자동 배포 - VM = 컨테이너 어플라이언스 모델 - Libvirt/OVS 네트워크 관계 트러블슈팅 - 이벤트/로그 분석 - VM 마이그레이션 - 장애 시나리오 대응
--	---

3. OpenStack 서비스 설치(오픈스택 어드민)

교육명	OpenStack 서비스 설치(오픈스택 어드민)
수정	2025-12-26
교육개요	본 과정은 표준 오픈스택 플랫폼의 설치 및 기본 사용법을 학습하는 입문 과정입니다. 오픈스택의 핵심 구성 요소에 대한 이해를 바탕으로, 클라우드 인프라 환경을 직접 구축하고 운영할 수 있는 기초 역량을 다집니다. 또한, 컨테이너 기반으로 동작하는 오픈스택 컴포넌트의 구조를 이해하고, Kolla 및 Kayobe를 활용한 클러스터 프로비저닝 및 배포 과정을 실습을 통해 익힙니다.
목표	kolla-ansible 기반으로 오픈스택 클러스터를 설치하고 운영할 수 있다. Kayobe를 활용한 베어메탈/노드 프로비저닝 흐름을 이해한다. 기존 TripleO와 신규 배포 방식(Kolla/Kayobe)의 차이를 설명할 수 있다. 오픈스택 CLI를 활용하여 주요 리소스를 직접 생성·관리할 수 있다.
기간	총 5일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	최소 2년의 리눅스 운영 및 사용 경험이 있는 엔지니어 혹은 사용자 아마존 서비스에서 온프레미스 서비스로 환경을 옮겨오려는 서비스 설계자 혹은 엔지니어
1일	오픈스택 과정 및 랩 소개 - 랩 구성 및 목적 오픈스택 역사 및 OpenInfra - 오픈스택의 시작 - 오픈스택과 OpenInfra 그리고 CNCF - 현재 오픈스택의 방향과 목적 설치를 위한 랩 준비 및 구성 - 오프라인 설치를 위한 설명 - Product vs PoC 모델 - 운영 및 구성을 위한 기술 오픈스택 기본 컴포넌트 소개 - Keystone, Nova, Neutron, Cinder, Heat, Glance, Ceilometer 대표적인 확장 컴포넌트 소개 - Octavia, Designate... 오픈스택 CLI 및 Dashboard(horizon) 확인하기 차세대 UI Skyline과 horizon의 차이점 오픈스택 설치 - 오픈스택 설치가 가능한 배포판 - 오픈스택과 SELinux 관계 - 클러스터에서 사용할 임시 스토리지 구성

	- kolla-ansible 기반으로 오픈스택 설치
2일	오픈스택 설치 - Podman vs Docker - Bifrost와 Director의 차이점 - kolla-ansible와 openstack-ansible 차이점 그리고 활용. - 기존 오픈스택과 달라진 설정 및 로그 파일 구성 - 플레이북 설정 및 설치 오픈스택 keystone 자원 다루기 - 사용자/프로젝트/역할/그룹 생성 및 관리 - 엔드포인트/카달로그와 오픈스택 서비스 관계(service project) - 서비스 계정과 일반 계정의 차이점 - 사용자/프로젝트/그룹/역할 활용하여 계정 구성 오픈스택 이미지 - 오픈스택에서 말하는 이미지(컨테이너 및 가상머신) - 이미지 빌드를 위한 도구 및 방법 - guestfs-tools와 libvirt 관계 - 이미지 빌드 및 업로드
3일	오픈스택 네트워크 - 오픈스택에서 제공하는 SDN 유형 - OVS/OVN 그리고 Appliance Network의 차이 ■ OVN의 장점과 한계점 ■ Openvswitch는 왜 사용하는가? ■ Linuxbridge와 OVS의 차이점 및 관계 - Provider/Flat/Tenant Network 구성 - VLAN 통한 Provider 네트워크 구성 - 오픈스택 네트워크/서브넷 생성 ■ 네트워크 생성 ■ 서브넷 생성 ■ 라우터 구성 ■ FloatingIP와 IP Address Pairing ■ nftables/OVS/OVN 사용 시, 자원 구성 차이 - 오픈스택 외부 네트워크 생성 ■ 오픈스택 외부 네트워크 구성 - * Provider/External 네트워크 차이
4일	오픈스택 블록 - 블록 스토리지 서비스 설명 - 블록 스토리지 백-엔드와 드라이버 타입 - 백-엔드 드라이버 설명

	■ 왜, LVM2는 진짜로 사용하면 안 되는가? ■ NFS드라이버는 무엇으로? (NFS vs Ganesha) ■ 이외 확장한 가능한 스토리지 - 블록 장치 생성 및 할당 그리고 확인 오픈스택 미터링 서비스 - 미터링 서비스 설명 - 미터링 서비스를 통한 서비스 사용량 확인 - 미터링 서비스는 어렵다! ■ Gnocchi 서비스 설명 및 확인 ■ Panko 서비스 설명 및 확인 ■ Adho 서비스 설명 및 확인 - 이제는 미터링 그만! - * 현대적으로 운영 및 구성하기 위한 Prometheus, Grafana 운영.
5일	오픈스택 가상머신 - 오픈스택 nova와 nova-api 그리고 nova-conductor의 관계 및 차이점 - 가상머신 구성을 위한 준비 ■ Flavor ■ Security Group ■ Firewall와 Security Group의 차이점 그리고 OVN ■ 가상머신 관리를 위한 명령어 훑어보기 ■ libvirt, sasl관계 그리고, 마이그레이션 ■ libvirt디버깅을 위한 접근 방법 - 가상머신 생성 ■ 가상머신 콘솔에 접근하기 ■ 사용이 가능한 콘솔 유형 - 외부에서 접근 가능하도록 FloatingIP할당 오픈스택 자동화 - Heat 서비스 설명 ■ Ansible과 Heat의 차이점. ■ 왜 둘 다 YAML기반으로 사용하는가? - Heat 문법 설명 - Heat 기반으로 간단한 서비스 배포 오픈스택 서비스 확장 - 서비스 확장을 위한 플레이북 훑어보기 - 서비스 확장 및 확인 - 오픈스택에서 CI/CD 오픈스택 기반 쿠버네티스 구성 및 구축 - 구축 시 필요한 컴포넌트

	- 오픈스택 오퍼레이터 - 이해가 필요한 확장 기술 - 쿠버네티스 클러스터 구성
--	--

4. 쿠버네티스 기업용 가상화 하베스터 운영

교육명	쿠버네티스 기업용 가상화 하베스터 운영
수정	
교육개요	Harvester는 Rancher가 주도한 오픈소스 HCI(Hyper-Converged Infrastructure) 플랫폼으로, KVM 기반 가상화와 Kubernetes 통합 환경을 제공한다. 본 교육 과정은 Harvester 설치부터 VM 배포, 네트워크 구성, Rancher 연동, 멀티 노드 고가용성 구성 및 Kubernetes 통합 운영까지 실무 중심으로 구성되며, 글로벌 트렌드에 따라 GitOps 및 클라우드 인프라 자동화도 함께 실습한다. 본 과정은 VMware 중심 가상화 환경에서 Kubernetes 중심 HCI로 전환하고자 하는 조직을 대상으로, 운영 관점에서 Harvester를 안정적으로 설계·배포·확장하는 방법을 실습 중심으로 학습한다.
목표	- Harvester의 구조 및 작동 원리 이해 - 하이퍼컨버지드 VM 환경 직접 구축 및 운영 - Harvester 내 PXE, ISO, 이미지 저장소 구성 - Rancher 연동 및 RKE2 클러스터 배포 - 멀티 노드 HCI 고가용성 구성 - VM 기반 워크로드와 K8s 네이티브 워크로드 통합 운용
기간	총 5일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구 - 최소 5대 이상의 노드(VM 또는 실 서버) 구성 - PXE + DHCP + TFTP + HTTP 부트 환경 - 외부 DNS, NFS, VM 이미지 저장소, Rancher 설치 환경
교육대상	Kubernetes 기반 HCI 도입 또는 전환을 검토 중인 인프라 관리자 VMware / KVM / Proxmox 환경에서 쿠버네티스 중심 가상화로 확장하고자 하는 실무 엔지니어 Rancher, RKE2 기반 멀티 클러스터 및 VM-K8s 혼합 운영을 담당하는 운영자 PXE, 네트워크, 스토리지 기반 인프라를 이해하고 운영 자동화(GitOps)에 관심 있는 DevOps 엔지니어
1일	Harvester 이해 및 설치 환경 준비 - Harvester 개요 및 아키텍처 소개 - HCI와 전통 가상화(KVM, Proxmox 등)의 차이 - PXE 부팅 구조: DHCP/TFTP/HTTP 구성 원리 - ISO, Cloud-Init, 이미지 저장소 구조 설명 실습

	PXE 환경 준비 (dnsmasq + httpd + TFTP) Harvester ISO PXE 자동 설치 준비 Harvester 3~5노드 클러스터 설치 실습
2일	2일차 – VM 인프라 구성 및 스토리지 연동 - Harvester 내 VM 생성 방식 및 Cloud-Init 이해 - Virtual IP, VLAN, 브리지 네트워크 구성 - 내장 Longhorn 스토리지 구조 이해 실습 Harvester 기반 VM 생성 (Cloud-Init, ISO) ISO/이미지 업로드 및 템플릿 구성 Longhorn 상태 확인 및 볼륨 관리 실습
3일	Rancher 연동 및 RKE2 클러스터 통합 - Harvester ↔ Rancher 통합 구조 설명 - VM 워크로드에서 K8s 워크로드로 확장 - RKE2 클러스터 설치 방식 소개 실습 Rancher 설치 및 Harvester 클러스터 연결 Harvester UI에서 RKE2 클러스터 생성 생성된 RKE2 클러스터에 GitOps 앱 배포
4일	네트워크 고가용성 정책 - Multus/CNI 연동 구조 설명 - LoadBalancer, Ingress 구성 - 네트워크 격리 및 보안 정책 구성 - HA 구성 요소 (ETCD, Harvester HA 구조) 실습 VM에 Static IP 설정 및 외부 통신 확인 RKE2 + Harvester 연계 Ingress 서비스 배포 스토리지 장애/네트워크 장애 복구 실습
5일	GitOps 및 운영 자동화 - VM 배포 자동화 전략 (Cloud-Init, REST API, Salt 등) - GitOps 연계 (ArgoCD + Rancher Fleet) - 모니터링/알림/로깅 연동 전략 실습 Rancher Fleet 기반 GitOps 구성 Git 푸시 → RKE2 + VM 환경 자동 반영 Prometheus + Grafana를 통한 VM/K8s 통합 모니터링 알림: Alertmanager 또는 웹훅 기반 Slack 연동

자동화&Dev/MLOPs

1. 앤서블 코어 기반 인프라 자동화

교육명	앤서블 코어 기반 인프라 자동화
수정	2025-12-23
교육개요	본 과정은 Ansible의 기본 개념부터 플레이북 작성, 인벤토리 구성, 롤(Role), Ansible Galaxy, 모듈의 실제 동작 방식까지 실무에 필요한 전 영역을 학습합니다. 또한 OpenStack·Kubernetes·리눅스 운영 환경에서 적용할 수 있는 실제 자동화 시나리오 기반으로 실습을 진행합니다. 초급 엔지니어 및 자동화 입문자를 대상으로 하며, 운영 자동화 IaC(Infrastructure as Code)의 관점에서 Ansible을 체계적으로 익힌다.
목표	Ansible의 구조 및 작동 방식 이해 Inventory/Playbook/Role/Module 사용법 숙달 Ansible을 활용한 시스템 구성 자동화 능력 습득 OpenStack·Kubernetes·Linux 운영 자동화 실습 실제 운영 환경에서의 트러블슈팅 및 모범 사례 실습
기간	4일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	Ansible을 활용하여 Linux/Windows 시스템 자동화를 시작하려는 엔지니어 기존 Shell/Python 스크립트 기반 운영 작업을 Ansible Playbook으로 전환하고자 하는 실무자 Role 기반 구조로 반복·대규모 운영 작업을 표준화하고 싶은 인프라 엔지니어
1일	Ansible 기초와 작동 구조 Ansible 소개 - Ansible 철학: Agentless/Push Model - YAML 기본 문법 - Control Node와 Managed Node 구조 - SSH 기반 자동화 흐름 설치 및 환경 구성 - Ansible 설치 - ansible-config, ansible-doc 활용 - 인벤토리 작성 - Ad-hoc 명령 실습 ■ ping, shell, copy, file 등 기본 모듈 Ansible 아키텍처 이해 - ansible.cfg - Inventory 구조(ini, yaml, dynamic inventory 소개) - Hostvars / Groupvars

	- 변수 우선순위 실습 - 첫 번째 인벤토리 구성 - 3개 이상의 노드를 대상으로 Ad-hoc 명령 실행 - 파일 배포 및 사용자 생성 자동화
2일	Playbook 작성 - Play / Task / Handler / Block - when / loop / register / ignore_errors - Template(Jinja2) 이해 및 작성 - Error Handling 전략 Module 활용 - Core modules: copy, template, service, package - Filesystem modules - System modules - community.general, ansible.posix 등 컬렉션 활용 Ansible Facts - setup 모듈 - 커스텀 변수로 facts 활용 - condition과 facts 조합 실습 Web 서버 자동 배포 Playbook 작성 Template 기반 설정 적용 서비스 자동 시작 + notify/handlers 구성
3일	Role / Ansible Galaxy / 실무 자동화 Role 구조 - roles 디렉터리 구조 - defaults / vars / handlers - tasks/main.yml 구성 전략 - 역할 분리의 기준 Ansible Galaxy - Role 다운로드 - Collections 구조 - requirements.yml 활용 운영 자동화 시나리오 - Linux 초기 설정 자동화 (user, ssh, sysctl, firewalld 등) - Nginx/Apache 자동 배포 - 로그 수집/배포 자동화 - 백업 스크립트 자동 배포

	실습 Role 직접 생성 웹 서비스 Role + DB Role 조합 tags 기반 실행 멥등성(idempotency) 검증
4일	OpenStack / Kubernetes 연동 및 운영 자동화 OpenStack 자동화 - openstack.cloud 컬렉션 소개 - identity_user / network / server 모듈 - VM 생성 / 삭제 자동화 Playbook - Floating IP / Security Group 관리 자동화 Kubernetes 자동화 - k8s.core 모듈 - Deployment / Service / Ingress 관리 - Helm Chart 배포 자동화 - 쿠버네티스 클러스터의 선언형 자동화 대규모 운영 자동화 전략 - Dynamic Inventory - GitOps 연동 - CI/CD에서 Ansible 활용 - Molecule 기반 테스트 소개 종합 실습 - “서비스 전체 자동화” 과제 ■ VM 생성 → 패키지 설치 → 앱 배포 → 상태 점검 - 문제 상황 재현 & 트러블슈팅 - 멥등성 검증 - 실무 배포 전략 공유

2. OpenTofu(테라폼 오픈소스)

교육명	OpenTofu(오픈토포, 오픈소스)
수정	
교육개요	2023년 이후 HashiCorp는 Terraform을 포함한 대부분의 제품 라이선스를 MPL 2.0 → BSL(Business Source License) 로 변경하였습니다. 다음과 같은 조건으로 반영이 되어 있습니다. ■ BSL은 오픈소스가 아님 ■ 상업적 SaaS 제공, 제품 통합, OEM 형태 활용에 강한 제약 ■ 대규모 인프라/플랫폼팀은 법적 리스크 없이 자유롭게 쓰기 어려움 ■ 특히 공공기관·교육기관·엔터프라이즈 자동화 환경에서는 BSL을 채택하

	기 어려움 ■ OpenTofu는 완전한 오픈소스(Community-Driven fork)로 탄생 ■ 기업·교육·연구 환경에서 가장 안전한 선택 이러한 이유로, 테라폼의 오픈소스 버전인 오픈토푸의 기본적인 문법 및 활용 방법에 대해서 학습합니다. 현재 리눅스 파운데이션은 공식적으로 오픈토푸를 기본 자동화 도구로 후원 및 지원하고 있습니다.
목표	- OpenTofu 문법 및 기본적인 사용 방법 - OpenTofu 아키텍처 이해 - 운영체제 애플리케이션 설치 및 설정 - 하드웨어(디스크·및·네트워크) 설정
기간	총· 4일·조정가능·과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	DevOps 초급자, 인프라 엔지니어, 클라우드 초급 관리자 등 기존 테라폼 사용자가 오픈소스 버전으로 전환
1일	IaC 개념 소개 및 OpenTofu 등장 배경 HCL(HashiCorp Configuration Language) 문법 이해 OpenTofu 설치 및 환경 구성 실습 첫 번째 .tf 파일 작성 및 리소스 배포 (local file 등)
2일	Provider 구조와 블록 이해 Variable과 Output 설정 실습 상태 관리(State)와 tofu state 분석 terraform.tfstate와 백엔드 개념
3일	모듈(Module) 구조와 재사용성 실습 Git 기반 모듈 관리 외부 변수 파일 활용 및 워크스페이스(Workspace) 실습 디버깅 및 로깅 (TF_LOG, plan, validate)
4일	OpenTofu를 활용한 AWS/OpenStack 리소스 실습 네트워크 구성, 가상 머신 배포 CI/CD 연동 개요 (예: GitHub Actions, GitLab CI 등과의 연결) OpenTofu + Ansible 연동 구조 소개

3. AI Assisted Tekton CI/CD(CPU 기반 SLM을 활용한 YAML 생성과 자동화)

교육명	AI Assisted Tekton CI/CD(CPU 기반 SLM을 활용한 YAML 생성과 자동화)
수정	2026-12-26
교육개요	본 과정은 Kubernetes 네이티브 CI/CD 도구인 Tekton을 기반으로, 선언적 파이프라인을 직접 설계·구현하고 이를 AI(SLM, Small Language Model)를 활용해 효율적으로 생성·보조하는 실무 중심 교육 과정입니다. Tekton YAML은 버전 변화와 구성 복잡도로 인해 반복적인 작성 부담이 크기 때문에, 본 과정에서는 CPU 기반 SLM을 활용한 Skeleton Code 생성 방식을 도입하여 운영자가 제어 가능한 범위 내에서 코드 작성을 단순화합니다. 특히 외부 API 호출이 불가능한 폐쇄망(내부망) 환경을 가정하여, SLM 런타임을 직접 구성하고 KIKI 에이전트를 통해 Tekton YAML을 생성·검증하는 현실적인 자동화 시나리오를 실습합니다. 이를 통해 수강생은 AI를 '대신 개발해주는 도구'가 아닌, CI/CD 운영 효율을 높이는 보조 자동화 도구로 활용하는 방법을 학습하게 됩니다
목표	• Tekton CI/CD 아키텍처 이해 • Tekton YAML 직접 설계 및 디버깅 • SLM 기반 코드 생성 활용 능력 • 폐쇄망 환경에서의 AI 활용 전략 이해 • KIKI 에이전트를 활용한 자동화 흐름 구성 • 현업 적용 가능한 CI/CD 자동화 시나리오 완성
기간	총 5일, 4일로 조정가능 과정
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구 GPU 사용이 가능한 노트북
교육대상	쿠버네티스 기본 개념 및 리소스(YAML)를 이해하고 있는 엔지니어 기존 Jenkins 기반 CI/CD를 운영 중이거나 전환을 고려하는 인프라/플랫폼 엔지니어 GitOps 또는 Kubernetes 네이티브 CI/CD에 관심 있는 DevOps 엔지니어 사내 폐쇄망 환경에서 CI/CD 표준 설계 및 운영해야 하는 기술 담당자
1일	랩 소개 - 오픈스택 기반 쿠버네티스 설치 및 테크톤 설치 - 자동화를 통한 쿠버네티스 설치 - 테크톤 설치 테크톤 탄생 및 소개 - 쿠버네티스 혹은 Cloud Native에서의 CI/CD입장 테크톤 기본 블록 문법 - "헬로 월드" 작성하기

	테크톤 태스크 설명
2일	테크톤 태스크 설명(계속) - 태스크(tasks) 및 스텝(steps) 이해하기 - 첫 태스크 작성하기 - 태스크 파라미터 구성 및 사용하기 - 데이터 공유(쿠버네티스 볼륨) 테크톤 대시보드 - 대시보드 설치 및 접근 태스크 디버깅 - tkn 명령어를 통한 자원 디버깅 - kubectrl 명령어를 통한 자원 디버깅 GIT/REGISTRY 서버 구성 - Gogs 기반으로 GIT OPs 서버 구축 - docker-registry 기반으로 컨테이너 레지스트리 서버 구성
3일	테크톤 파이프라인 작성하기 - 파이프라인 설명 - 첫 번째 파이프라인 작성 - 파이프라인 파라미터 - 파이프라인 순서 지정 및 "Pipeline runs"를 통한 결과 확인 워크스페이스 - 워크스페이스를 통한 데이터 공유 - 데이터 공유가 가능한 쿠버네티스 자원(storageclass,pv,pvc) - 파이프라인을 통한 영구자료(Persistent Data) - 워크스페이스 사용하기 - 워크스페이스 템플릿 생성
4일	조건식(operator) - 테크톤 조건식 설명 - 조건식 지시자 - 조건식 및 파라미터 인증 처리 - 테크톤 인증 처리 설명 - 기본 인증 - SSH 인증 - 컨테이너 레지스트리 인증
5일	트리거 - 테크톤 트리거 설명 및 설치 - 클러스터에 트리거 설정 - 트리거 템플릿/바인딩 및 이벤트 리스너

	- 트리거 템플릿 기반으로 트리거 생성 - 트리거 및 웹훅 ArgoCD와 연동 및 설정 - ArgoCD 설명 - ArgoCD 설치 - ArgoCD와 Tekton의 차이점 - ArgoCD와 테크톤 연동 및 배포 최종실습 - LocalLLM 기반 YAML 생성 → 테크톤 실행 → GitOps배포 - 폐쇄형 환경에서의 CI/CD 자동화 시나리오 완성
--	--

퍼블릭 클라우드

1. AWS 기본 서비스

교육명	아마존 기본 서비스
수정	
교육개요	대표적인 퍼블릭 클라우드 서비스인 아마존 AWS기반으로 어떠한 서비스를 구성 및 구축할 수 있는지 확인한다. 이를 통해서 데이터센터에서 구성하는 인프라와 퍼블릭 클라우드의 차이점을 확인한다.
목표	아마존 AWS 기본적인 서비스를 이해한다. IDC에 구성하는 시스템과 어떠한 차이점이 있는지 이해한다. 기본적인 서비스 이해를 한다.
기간	총 4일
랩	아마존 계정 필요
교육대상	1년 미만 리눅스 사용경험이 있는 엔지니어 혹은 개발자 아마존 퍼블릭 클라우드 서비스에 대해서 관심이 있는 엔지니어 혹은 개발자
1일	클라우드 및 AWS소개 - 클라우드 개념 및 AWS서비스 소개 - 서비스 가입과 웹 콘솔 사용 아마존 기본 보안 - IAM 서비스로 사용자 계정 관리 - 권한 정의 및 제어(RBAC)
2일	네트워크 - VPC 기반으로 망 분리(Public/Private) - 방화벽으로 트래픽 제어 - 고정 IP로 지속적인 통신 구현 컴퓨팅 서비스 - EC2 가상머신 서비스 - 인스턴스 유형과 비용 - 가상머신 연결 및 작업 - 외부에서 서비스 요청
3일	AWS Storage S3 - Simple Storage Service -S3 - Buket으로 데이터 저장 - 정적 웹 호스팅 서비스 구현 - Storage Class(Glacier 서비스 통합) AWS EBS/EFS - 블록 스토리지 구현 - 공유 스토리지 기반으로 데이터 공유

4일	Serverless 시스템 - Serverless Platform 소개 - Lambda서비스 운영 자동화 데이터베이스 - 다양한 데이터베이스 서비스 - RDS기반으로 데이터베이스 구성 및 설정
5일	모니터링 - CloudWatch로 모니터링 - Metric 및 Alarm서비스 서비스 스케일링 - 오토 스케일링 서비스 - ELB(로드 밸런서)와 부하 분산 - 고가용성(High Availability) 서비스 구성

네트워크

1. OVS/OVN 기반 인프라 엔지니어 네트워크 실무 교육 커리큘럼

교육명	OVS/OVN 기반 인프라 네트워크 실무 마스터 과정
수정	
교육개요	OVS 및 OVN의 내부 구조와 사용법을 중심으로 OpenStack 및 Kubernetes 환경에서의 SDN 네트워크 구성, 보안 제어, 트러블슈팅까지 실습 중심으로 다루는 과정
목표	- OVS의 구조 및 구성법 이해 - OVN의 L2/L3 구성 및 정책 설정 - OpenStack과 OVN의 통합 구성 실습 - Kubernetes-OVN 연동을 통한 CNI 제어 이해 - ACL, NAT, Flow 분석 기반 트러블슈팅 역량 확보
기간	총 5일, 4일 조정가능. 다만, kubernetes-ovn 제외
랩	가상화 기반 실습 환경 (OpenStack VM 기반 OVN 구성 / Kubernetes 클러스터 + OVN-CNI)
교육대상	네트워크 인프라 엔지니어, OpenStack/Kubernetes 운영자, 클라우드 네트워크 설계자
1일	OVS 구조 및 VLAN 실습 - OVS vs Linux Bridge 구조 비교 - Bridge/Port/VLAN/Bonding 구성 실습 - 주요 명령어: 'ovs-vsctl', 'ovs-ofctl' Mini Lab: - 'br0' 브릿지를 만들고 eth1 포트를 추가

	- VLAN 10, 20 설정 후 VM 간 통신 테스트 - **VM 실습:** - VM1 (vlan10): 192.168.10.10 - VM2 (vlan20): 192.168.20.10 - - ping 테스트로 브로드캐스트 도달 확인
	OVN 논리 네트워크 구성 - OVN 아키텍처 이해 (northd, NB/SB DB) - Logical Switch, Logical Router 실습 - ACL 정책 기본 구조 구성 Mini Lab - `ls0`, `lr0` 생성 및 포트 연결 - ACL로 ping 차단 후 `ovn-trace` 확인 VM 실습 - VM1: Logical Switch A - VM2: Logical Switch B - - 라우터 연결로 통신 시도 후 ACL 적용 여부 확인
	Geneve 터널 구성 및 MTU 확인 - NAT(SNAT/DNAT) 흐름 및 Floating IP 이해 - 흐름 분석 도구: `ovn-trace`, `ovn-sbctl` Mini Lab - 외부 연결을 위한 NAT 구성 - `ovn-trace`로 8.8.8.8 ping 트래픽 흐름 분석 VM 실습 - 내부 VM에서 외부로 ping 후 Floating IP 적용 - - 외부에서 VM으로 ssh 시도 → DNAT 검증
	Day 4: ACL 고급 정책 및 장애 처리 - ACL 우선순위와 예외 규칙 구성 - 장애 시 흐름 차단/허용 정책 실습 - 정책 오류 및 흐름 복원 실습 Mini Lab - 특정 IP에 대해서만 HTTP 허용 ACL 구성 - 정책 우선순위에 따라 흐름 차단 테스트 VM 실습 - VM1: 허용 IP - VM2: 차단 대상 - - curl로 통신 여부 확인 및 로그 분석
	Kubernetes OVN 연동 - OVN-Kubernetes 구성 요소 이해

	- OVN-K8s pod ↔ pod, pod ↔ external 흐름 분석 - K8s NetworkPolicy 구성 실습 Mini Lab - allow-http NetworkPolicy 적용 후 결과 비교 - pod 간 통신과 외부 연결 차단 여부 확인 VM 실습 - pod-A: frontend (80 open) - pod-B: backend (deny all) - - `kubectl exec`로 HTTP curl 확인
--	--

스토리지

2. 오픈소스 분산 스토리지 CEPH 기본

교육명	오픈소스 분산 스토리지 CEPH 기본
수정	2025-12-23
교육개요	본 과정은 오픈소스 분산 스토리지 표준으로 자리 잡은 Ceph 스토리지의 구조, 설치, 운영 방법을 실습 중심으로 학습하는 과정입니다. 기존 OpenStack 환경에서 사용되던 Swift는 현재 활용 범위가 제한적이며, 이에 따라 Ceph Object, Block, File 스토리지는 기업 환경에서 사실상 표준 스토리지로 활용되고 있습니다. 본 교육에서는 Ceph의 핵심 구성 요소(OSD, MON, MDS), 데이터 배치 알고리즘 (CRUSH), Object Storage 복제 구조, CephFS 활용 방식 등을 중심으로 실제 운영 환경에서 필요한 설계와 관리 방법을 학습합니다.
목표	Ceph 스토리지 설치 OSD/MDS/MON 그리고 Ceph스토리지 아키텍처 확인 스토리지 인증 및 사용자 관리 Object/CephFS(fuse) 직접 구현 및 활용
기간	총 4일, 3일로 조정은 가능하나, 몇 교육 모듈은 제외해야 됨.
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	CEPH에 관심있는 스토리지 엔지니어 클라우드 시스템에 오브젝트 스토리지 구성이 필요한 사용자 Swift에서 Ceph 스토리지로 전환을 고려하는 사용자 Longhorn과 Ceph 스토리지 차이점을 확인하고 싶은 스토리지 엔지니어
1일	Ceph 개념 및 아키텍처 - 분산 스토리지의 장점 - Ceph에서 제공하는 CephFS - Ceph알고리즘(rados, crush) - 클러스터 설계 규칙 - Ceph 스토리지 클러스터 설치 및 초기 구성

	- OSD/MON/MDS 아키텍처 이해 - Ceph 사용자 및 접근 제어 정책 이해 - Object Storage 및 CephFS(FUSE) 직접 구성 및 활용 - OpenStack/Kubernetes 연동 구조 이해 Ceph 설치 - 저장소 살펴보기(배포판 별) - 패키지 설치 ■ Ceph RPM 혹은 DEB ■ cephadm 도구와 앤서블 관계 ■ Ceph Cluster 설치 - Ceph에서 제공하는 API ■ 쿠버네티스에서 Ceph ■ 오픈스택에서 Ceph - 쿠버네티스 기반으로 Ceph 설치 ■ * okd에서 ceph스토리지 구성(데모, 실습 없음)
2일	Ceph 자원 단위 - Ceph PG - PG계산 방식 및 적용 - Ceph Pool과 PG 관계 - CRUSH 설정 - OSD, MDS, MON의 기능 및 설명 Ceph 명령어 - 기본적인 명령어 사용 방법 ■ 클러스터 자원 조회 ■ 클러스터 OSD자원 조회 ■ 클러스터 pool자원 조회 Ceph 사용자 - Ceph 사용자 생성 및 관리 - Ceph에서 제공하는 권한/접근 제어 정책 Ceph Pool - Ceph Pool개념 - Pool과 OSD의 관계 OSD 확장하기 - OSD 확인 - OSD 확장 및 확인 OSD 기반으로 자원 관리 - OSD 기반 자원 관리 (업로드/조회 등) - - 자원 조회 및 접근

3일	Ceph 블록 스토리지 - RADOS Block Device 설명 ■ POSIX 호환성은 무엇인가? - OSD와 RADOS Block Device 차이점 - RADOS기반으로 장치 생성 및 구성 - RADOS와 CephFS의 연관 구조 이해 ■ CephFS의 FUSE 기반 마운트 방식 이해 ■ FuseFS와 Ceph관계 ■ MDS와 OSD/MON 그리고 RADSO ■ CephFS기반으로 블록장치 구성 및 연결 - RADOS Gateway ■ RADOS 게이트웨이 설명 - * RADOS기반으로 자원 활용
4일	스토리지 통합 - 오픈스택에서 Ceph스토리지 사용 - 쿠버네티스에서 Ceph스토리지 사용 모니터링 - 클러스터 상태 모니터링 튜닝 - 수정 가능한 커널 파라미터 - 수정 가능한 네트워크 파라미터 - 수정 가능한 Ceph Server/Client

3. 분산 NAS GlusterFS 클러스터 구성 및 운영

교육명	분산 NAS GlusterFS 클러스터 구성 및 운영
수정	2025-12-23
교육개요	GlusterFS는 최신 트렌드의 스토리지는 아니지만, NFS 기반 워크로드, 혼합 인프라(OpenStack + Kubernetes + BareMetal) 환경에서는 여전히 단순하고 안정적인 SDS 솔루션입니다. 본 과정은 GlusterFS를 <i>대체 불가능한 상황</i>에서 어떻게 설계하고, 어디까지 쓰고 어디서 멈춰야 하는지를 실습 중심으로 학습을 진행합니다.
목표	GlusterFS 클러스터를 직접 설계·구성·운영할 수 있다 OpenStack/Kubernetes/BareMetal 환경에서 어느 부분까지 적용이 가능한지 확인이 가능하다. 장애 상황(split-brain, brick 손실)에 대해 복구 시나리오를 이해하고 대응할 수 있다 기존 NFS와 어떠한 차이점이 있는지 확인이 가능하다.
기간	총 3일, 4일로 권장

랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	SDS 스토리지 엔지니어 Kubernetes 및 OpenStack에서 NAS 스토리지 구성 NFS4기반의 pNFS 시스템이 필요한 스토리지 엔지니어
1일	GlusterFS 무엇인가? - GlusterFS 아키텍처 - GlusterFS에서 사용하는 표준 오픈소스 컴포넌트 - GlusterFS와 NFS/CephFS/GFS2의 차이점 - 다른 NAS와 GlusterFS와 비교 GlusterFS 설치 - 사용가능한 운영체제 확인 - 설치방법 및 구성 GlusterFS 명령어 - - gluster 명령어 사용
2일	블록 생성 및 구성 - GlusterFS에서 제공하는 Brick/Volume 그리고 GlusterFS Daemon - 기본 볼륨 및 블록 생성하기 - 볼륨 디스크 연결 및 구성 볼륨 관리 - 구성된 볼륨 기반으로 복제 및 백업 - 볼륨 확장 및 축소, 그리고 리밸런싱 브릭/블록 복제 - 볼륨 복제 설명 - Geo Replication 복제 - Geo Replication 확인 볼륨 쿼터 - 볼륨 쿼터 설명 - - 볼륨 쿼터 설정 및 확인
3일	브릭 추가 및 제거 - 브릭과 노드의 관계 - 노드추가 및 브릭 확인 - 노드제거 및 브릭 확인 배포 및 복제 설정 - 클러스터 노드 복제 구성 ■ 배포 브릭 구성 - 분산 브릭 구성 클러스터 모니터링 웹 관리자 페이지 접근

4일	고급 기능 및 컨테이너 연동 - GlusterFS 성능 튜닝 - 캐시, ping-timeout 등 성능 옵션 확인 및 설정 - 장애 복구 실습 - split-brain 재현 및 해결 - brick 삭제 후 복구 실습 Kubernetes 연동 - Heketi 구성 - Kubernetes StorageClass 생성 및 PVC 테스트 RESTful API 구성 - Heketi API 연동 실습 - - CLI 외 볼륨 생성 자동화
----	---

언어

1. LLM 에이전트 작성을 위한 Go 언어 기초

교육명	LLM 에이전트 작성을 위한 Go 언어 기초
수정	2025-12-23
교육개요	본 과정은 LLM 기반 에이전트 구조를 이해하고, 이를 실제로 동작하는 경량 Go 에이전트로 구현하는 실습 중심 과정입니다. Go 언어의 기본 문법, 병행성 모델, HTTP 서버 구현을 통해 LLM API 연동, 도구 실행, 상태 관리, API 기반 제어를 직접 구현하며, AI/ML 및 인프라 엔지니어가 운영 환경에서 활용 가능한 LLM 에이전트의 기초 구조를 완성하는 교육입니다. “쿠버네티스 컨트롤러, 오픈스택 서비스, 에이전트 기반 자동화는 전부 이 구조에서 출발합니다.”
목표	Go 언어 기반 LLM 에이전트 구조 이해 LLM API(OpenAI 호환 등) 호출 및 응답 처리 에이전트 내부 도구 실행 구조 구현 고루틴 기반 병렬 작업 처리 CLI + HTTP 인터페이스를 갖는 에이전트 완성
기간	4일 (총 24시간, 1일 6시간 기준)
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	AI/ML 엔지니어 (모델 서빙, 파이프라인 자동화 관심) 인프라/SRE/DevOps 엔지니어 Kubernetes, OpenStack, 시스템 자동화를 다루는 엔지니어 Python/Bash는 써봤지만 운영용 언어가 필요한 분
1일	에이전트의 뼈대 - Go 개발환경 구성

	- Go 기본 문법 - CLI 기반 에이전트 골격 생성 - 에이전트 명령 입력 구조 설계 미니 프로젝트: • LLM 에이전트 CLI 프레임워크 생성
2일	상태와 메모리 - 구조체로 Agent/Tool/Task 정의 - 파일 기반 상태 저장 (Memory, Context) - JSON 직렬화 미니 프로젝트: - 에이전트 상태 및 실행 이력 저장
3일	병렬 실행과 도구 - 고루틴, 채널, select - 병렬 도구 실행 - LLM 응답 기반 분기 처리 미니 프로젝트: - 병렬 Tool 실행 에이전트
4일	LLM 연동 & API - net/http 기반 서버 - LLM API 연동 (OpenAI 호환) - REST API로 에이전트 제어 HTTP 기반 LLM 에이전트 완성 • POST /agent/run • GET /agent/status

2. 인프라 엔지니어를 위한 MicroJava(Quarkus) 기초

교육명	인프라 엔지니어를 위한 MicroJava(Quarkus) 기초
수정	2025-12-26
교육개요	클라우드 환경에 최적화된 경량 자바 프레임워크 Quarkus를 기반으로, 인프라 엔지니어가 반드시 알아야 할 애플리케이션 구조, 설정 주입 방식, 헬스체크, 관측, 배포 자동화를 실습 중심으로 학습합니다. 소스 코드 구현보다 - "왜 이 설정이 필요한가" - "운영 시 어디를 봐야 하는가" - "문제 발생 시 어떻게 추적하는가" 에 초점을 둡니다.
목표	자바 마이크로서비스의 구조와 동작 방식 이해 Quarkus 애플리케이션의 설정·상태·헬스·메트릭 포인트 파악

	Kubernetes에서의 배포·운영·관측 실습 JVM vs GraalVM 네이티브 빌드의 운영적 차이 이해 CI/CD, 보안, 회복성 패턴을 운영 자동화 관점에서 연결
기간	3일 또는 4일 과정 (옵션에 따라 선택 가능)
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	Kubernetes 환경에서 자바 기반 서비스 운영을 담당하는 인프라 엔지니어 DevOps/SRE 역할을 수행하거나 준비 중인 엔지니어 "개발자는 아니지만 YAML과 로그, 메트릭은 내 책임"인 분들 JVM 기반 서비스가 왜 무겁고, Quarkus가 왜 다른지 알고 싶은 분들
1일	[Quarkus 구조 & 실행 모델 이해] - Quarkus 아키텍처: Build Time vs Run Time - JVM 모드 vs Native 모드 운영 차이 - Dev Mode, Dev UI로 내부 상태 관찰 - REST API 구조를 통해 엔드포인트·포트·컨텍스트 파악 - application.properties를 통한 설정 외부화 전략 - Live Reload가 운영에 주는 의미
2일	[관측 가능성(Observability) 집중] - PostgreSQL 연동 구조 이해 (코드보단 연결 흐름) - 트랜잭션과 커넥션 풀 관점에서의 병목 포인트 - Health/Readiness/Liveness 차이와 활용 - Metrics 구조 이해 (JVM vs Native) - OpenTelemetry 기반 Trace 흐름 추적 - Prometheus + Grafana 대시보드 실습
3일	[배포·설정·자동화] - Quarkus + Kubernetes 통합 구조 - ConfigMap/Secret 주입 패턴 - 환경변수 변경 시 재배포 전략 - GraalVM 네이티브 이미지 빌드 비교 실습 - 리소스 사용량(JVM vs Native) 관측 - Deployment YAML 자동 생성 흐름 - minikube 기반 실습 배포 - 미니 프로젝트: "운영 가능한 도서 API"
4일	[운영 자동화 & 안정성] - Tekton 또는 GitHub Actions 기반 빌드 자동화 - 이미지 빌드 → 배포 파이프라인 흐름 - Keycloak 연계 인증 구조 이해 - Retry/Timeout/Fallback의 운영적 의미 - Trace + Metrics + Logs 통합 관측 시나리오 - 미니 프로젝트 마무리 & 장애 시나리오 리뷰

3. 인프라 엔지니어를 위한 파이썬 활용

교육명	인프라 엔지니어를 위한 파이썬 활용
수정	2025-12-23
교육개요	인프라 운영 및 시스템 자동화를 위한 파이썬 언어 기초 교육으로, 반복 작업 자동화와 함께 LLM API를 활용한 간단한 에이전트형 스크립트 작성 경험 을 제공합니다.
목표	파이썬 문법과 구조 이해 시스템/네트워크 제어 자동화 스크립트 작성 API 및 로그 분석 실습 수행 LLM API를 활용한 간단한 자동화 에이전트 스크립트 이해 및 실습
기간	4일 (1일 6시간 기준, 총 24시간)
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	리눅스 기반 시스템 운영 경험이 있으며 자동화에 관심 있는 인프라 엔지니어 API, 로그, 모니터링 데이터를 코드로 다루는 기초 역량을 키우고 싶은 실무자 LLM을 "개발 대상"이 아닌 운영 보조 자동화 도구 로 이해하고 싶은 엔지니어
1일	파이썬 개요 및 설치 (VS Code, Jupyter 환경 구성) 변수, 자료형 (문자열, 리스트, 딕셔너리, 튜플, 셋) 조건문 (if, else, elif), 반복문 (for, while) 리스트 컴프리헨션 실습 실습: 사용자 입력을 받아서 정렬/필터하는 리스트 필터링기
2일	함수 선언, 반환값, 스코프 개념 모듈/패키지 import 방법 파일 입출력(open, with, read, write) 예외 처리 (try, except, finally) 실습: 로그 파일 분석기, 파일 백업 자동화 스크립트
3일	시스템 명령어 실행(os, subprocess 활용) CLI 인자 파싱(argparse) 디스크 사용량 점검, 네트워크 체크 자동화 실습: ping, traceroute 기반 네트워크 상태 수집기 실습: 파일 정리 스크립트 (확장자별 자동 분류)
4일	REST API 기본 (requests, json) Prometheus, Telegram 등 연동 실습 JSON 응답 처리 및 조건 필터링 LLM API 개념 이해 (Chat Completion/Prompt 개념) requests 기반 LLM API 호출 구조 이해 시스템 정보 또는 로그를 요약·분류·설명하는 간단한 스크립트 작성

	“자동 판단”이 아닌 보조 도구로서의 LLM 사용법 실습: 시스템 모니터링 자동화 및 경고 전송 종합 미니랩 (4개 중 선택 실습) 1. 파일 자동 정리 도구 2. 네트워크 점검 도구 3. 텔레그램 봇으로 경고 전송 4. LLM 에이전트 활용 • 로그 파일 일부를 LLM에 전달해 요약 출력 • 시스템 상태 결과를 자연어로 정리해서 출력 • 기존 스크립트 결과를 "사람이 읽기 좋은 문장"으로 변환
--	--

4. Modern Web Application with React & Kubernetes (인프라 엔지니어를 위한 React)

교육명	Modern Web Application with React & Kubernetes (인프라 엔지니어를 위한 React)
수정	2026-02-04
다이어그램	<pre>graph TD User[User / Browser] --> React[React Frontend (State / Component)] React --> API[API / BFF Layer (REST API)] API --> Backend[Backend Service] Backend --> Container[Container Image] Container --> Kubernetes[Kubernetes (Pod / Service)] Kubernetes --> Pipeline[CI/CD Pipeline (Build → Deploy)]</pre>
교육개요	본 과정은 HTML/CSS/JavaScript 및 기본적인 웹 개발 교육을 이수한 인프라 엔지니어 대상으로 React를 단순한 UI 라이브러리가 아닌 현대 웹 애플리케이션의 핵심 구성 요소로 이해하도록 설계된 실습 중심 교육 과정입니다. React 이전의 AJAX 기반 웹 개발 방식과의 비교를 통해 React의 등장 배경과 문제 해결 관점을 이해하고, React 기반 Frontend, API 기반 Backend, 그리고 이를

	연결하는 애플리케이션 구조를 직접 구성합니다. 마지막으로, 완성된 서비스를 컨테이너화하여 Kubernetes 환경에서 CI/CD 기반으로 배포함으로써 코드를 작성하는 인프라/소프트웨어 엔지니어가 서비스를 운영할 수 있는 서비스/소프트웨어 엔지니어 입장에서 관점 전환을 목표로 합니다.
목표	React가 등장하게 된 배경과 기존 AJAX 기반 웹 개발의 한계를 이해한다 React 기반 컴포넌트 및 상태 중심 애플리케이션 구조를 설계할 수 있다 Frontend-Backend 분리 구조의 웹 애플리케이션을 구성할 수 있다 간단한 테스트를 통해 애플리케이션 품질 관리 개념을 이해한다 CI/CD 파이프라인을 통해 React 서비스를 Kubernetes 환경에 배포할 수 있다
기간	3일
랩	
교육대상	HTML/CSS/JavaScript 기초 교육을 이수한 대학생 기본적인 웹 페이지 및 AJAX 통신 경험이 있는 학습자 React 및 최신 웹 서비스 구조를 처음 접하는 초급 개발자
1일	전통적인 웹 애플리케이션 구조 이해 - 서버 렌더링 중심 웹 구조 - jQuery/AJAX 기반 동적 화면 구성 방식 AJAX 기반 개발의 문제점 - DOM 직접 조작의 복잡성 - 상태 동기화 문제 - 코드 유지보수 난이도 증가 React의 등장 배경 - 선언적 UI 개념 - 컴포넌트 기반 설계 - 상태(State)와 화면 렌더링의 관계
2일	React 애플리케이션 구조 - 컴포넌트 분리 전략 - 상태 관리 기본 패턴 API 기반 Backend 구성 - 간단한 REST API 설계 - Frontend와 Backend 간 통신 구조 Middle Layer 개념 - BFF(Backend for Frontend) 개념 소개 테스트 개념 이해 - Frontend 테스트의 목적 - API 테스트 기본 개념 OCI 기반 컨테이너 생태계 이해

	- Podman/Docker의 차이점 - 오픈소스에서 OCI 그리고 레드햇/구글의 영향 - 이미지 빌드 도구(builder, buildx, podman) 차이 - Pod와 Container 차이점 - 애플리케이션 패키징 및 Pod 기반으로 배포
3일	서비스 배포 및 CI/CD 기반 운영 흐름 웹 애플리케이션의 배포 구조 이해 - 개발 환경 vs 운영 환경 컨테이너 기반 애플리케이션 패키징 - React 애플리케이션 컨테이너화 Kubernetes 기본 개념 - Pod, Service 개념 이해 - 프론트엔드 서비스 배포 구조 CI/CD 파이프라인 개요 - 소스 코드 변경 → 자동 빌드 → 자동 배포 흐름

모니터링

1. 오픈소스 통합 모니터링 운영 실무

교육명	오픈소스 통합 모니터링 운영 실무
수정	2025-12-26
교육개요	본 교육은 전통적인 서버 모니터링부터 클라우드 네이티브 환경의 메트릭 수집, 시각화, 관측성(Observability)까지 아우르는 오픈소스 기반 통합 모니터링 운영 과정입니다. Zabbix, Prometheus, Grafana, OpenTelemetry를 활용하여 단순 장애 감시를 넘어, 장애 원인 분석과 운영 기준선(Baseline) 설계까지 실무 중심으로 학습합니다. 특히 Datadog과 같은 상용 SaaS 없이도 온프레미스·폐쇄망 환경에서 구현 가능한 통합 모니터링 아키텍처를 직접 구성하고 운영해보는 것을 목표로 합니다.
목표	전통 인프라 및 클라우드 네이티브 환경의 모니터링 차이를 이해한다 Zabbix 기반 서버 및 네트워크 상태 모니터링을 수행할 수 있다 Prometheus 기반 메트릭 수집 및 Alert 설계를 수행할 수 있다 Grafana를 활용하여 운영자 관점의 대시보드를 설계할 수 있다 OpenTelemetry를 활용한 Observability 개념을 이해하고 적용할 수 있다 오픈소스 기반 통합 모니터링 아키텍처를 설계하고 운영할 수 있다
기간	5일
랩	오픈스택 기반 랩(총 5대 노드 사용 예정)
교육대상	인프라 엔지니어 시스템 관리자 클라우드/DevOps 운영자

	모니터링 및 운영 자동화에 관심 있는 실무자
1일	모니터링 개요 및 Zabbix 기반 전통 인프라 감시 - Monitoring vs Observability 개념 이해 - 장애 탐지와 장애 분석의 차이 - Zabbix 아키텍처 (Server/Agent/Proxy) - Active/Passive Check 구조 - Item/Trigger/Action 개념 - SNMP, ICMP, Agent 기반 모니터링
2일	Prometheus 기반 클라우드 네이티브 메트릭 수집 - Prometheus 설계 철학 (Pull 모델, Time Series) - Prometheus 아키텍처 이해 - Exporter 개념 및 구성 ■ node_exporter ■ blackbox_exporter - PromQL 기본 문법 및 패턴 - Alertmanager 개요
3일	Grafana를 활용한 시각화 및 운영 대시보드 - Grafana 역할 및 활용 시나리오 - Data Source 구성 (Prometheus, Zabbix) - 대시보드 설계 원칙 ■ 운영자용/관리자용/보고용 - Annotation 및 Alert 개념
4일	OpenTelemetry와 Observability 개념 - Observability 개념 이해 - OpenTelemetry 등장 배경 - OpenTelemetry 구성 요소 ■ Agent ■ Collector ■ Exporter - Metric/Trace 연계 구조 - Datadog 스타일 Observability의 오픈소스 구현
5일	통합 모니터링 아키텍처 및 실전 운영 - Zabbix와 Prometheus 역할 분리 전략 - 통합 모니터링 아키텍처 설계 - 장애 시나리오 기반 분석 흐름 - 운영 기준선(Baseline) 설정 방법 - 온프레미스/폐쇄망 운영 전략

세미나/부트캠프(이틀 이하 교육)

1. 하이브리드 클라우드를 위한 전략 부트캠프

교육명	하이브리드 클라우드를 위한 전략
수정	
교육개요	온프레미스와 퍼블릭 클라우드를 통합 운영하기 위한 전략적 접근 방법 및 기술 요소를 학습합니다. Kubernetes, Keycloak, MinIO, CI/CD 파이프라인을 중심으로 하이브리드 환경 구성 실습을 포함합니다.
목표	- 하이브리드 클라우드의 개념과 구성 요소 이해 - 인증 및 스토리지 통합 전략 습득 - 멀티 클라우드 환경과의 차이점 구분 - 실습을 통한 연동 구성 능력 배양
기간	2일
랩	오픈스택 기반 랩 및 퍼블릭 클라우드 계정(없어도 됨)
교육대상	인프라 관리자, DevOps 엔지니어, 클라우드 아키텍트, 하이브리드 인프라를 설계하고자 하는 실무자
1일	하이브리드 클라우드 개념 및 기본 개념 - 하이브리드 클라우드의 정의와 개념 소개(이 부분에서 우리는 Container/VirtualMachine 하이브리드로 초점) - 기업의 IT 인프라 변화에 대한 배경(퍼블릭 클라우드 및 컨테이너 기반의 인프라 시스템 영향) - 클라우드 서비스 모델(퍼블릭/프라이빗/하이브리드(멀티 클라우드와 동일한 뜻. 용어만 다름)) - IaaS/PaaS/SaaS의 특징 및 차이점 - Container/VirtualMachine 기반으로 하이브리드 클라우드
2일	클라우드 주요 도구 및 오케스트레이션 소개 - 표준 컨테이너 도구 소개 - 표준 가상머신 도구 소개 - API는 왜 중요한가? 시스템 엔지니어 입장에서 API - 하이브리드 클라우드 아키텍처 소개 및 충족조건 * 애플리케이션 및 데이터 포털 전략 - 온프레미스+퍼블릭클라우드=하이브리드? 멀티클라우드? - 만약, 하이브리드 클라우드 구현을 해야한다면...?

2. 표준 CI/CD(테크톤) 구축 및 활용 부트캠프

교육명	테크톤 세미나
교육개요	쿠버네티스와 테크톤을 함께 사용하여 빠르게 CI/CD 표준 인터페이스 구축한다. 이 교육에서 주요 목적은 설치가 아니, CI/CD 인터페이스 구축 배포 과정에 대해

	서 다룬다. 표준 컨테이너 개발 및 배포 환경에서는 Docker 및 Jenkins를 점점 사용하지 않는다. 대다수 자원은 YAML으로 선언 후 API기반으로 배포 및 구성이 되어야 한다. 이 세미나에서 이러한 구축을 사용자가 직접 구성 후, 서비스를 배포하여 NATIVE CICD환경에 대해서 더 쉽게 접근이 가능하도록 유도한다.
목표	쿠버네티스 기반으로 테크톤 서비스 구축 및 서비스를 배포한다.
기간	1~2일
코드	SEM-TKN-CICD
교육대상	모든 소프트웨어/시스템 엔지니어 및 기획자
1/2일	쿠버네티스 설명 - 오픈스택에서 쿠버네티스 사용 - AWS/OSP의 차이점 - 랩 접근 및 사양 확인 테크톤 - 테크톤 설명 - 테크톤에서 제공하는 자원 - 테크톤과 Jenkins의 차이점 - Jenkins X로 대체가 불가능한가? - 테크톤 명령어 학습 서비스 패키지 구성 - HelmChart 배포 - Operator 배포 - 테크톤과 위의 배포 방식의 차이점 - 어떠한 배포가 좋은 배포인가? CI/CD 구축 - GitTea 기반으로 GIT 및 Registry 서버 구축 - 간단하게 테스트용 코드 배포 - Podman과 Docker의 차이 - 이미지 빌드를 위한 buildah - 이미지 업로드 그리고, 관리를 위한 skopeo 명령어 - 테크톤을 통한 CI/CD 작업(tasks) 구성 - 파이프라인을 통해서 배포 시도 - Webhook를 통한 배포 자동화 서비스 배포 및 확인 K-NATIVE에 대한 설명 및 토론

3. Kube-virt/libvirtd 세미나

교육명	Kube-virt Libvirt 세미나
------------	-----------------------

교육개요	쿠버네티스는 컨테이너에서 가상머신 지원 및 제공한다. 쿠버네티스 기반으로 가상머신 배포 시, 어떠한 방법으로 이미지를 생성 및 배포해야 하는지 학습한다. 또한, CI/CD를 통해서 가상머신 이미지를 컨테이너 이미지처럼 자동 패키지/빌드/디플로이가 가능한지 확인한다.
목표	쿠버네티스 가상머신 기반으로 가상머신 이미지 빌드 및 배포에 대해서 학습한다.
기간	1일
코드	SEM-010
교육대상	모든 소프트웨어/시스템 엔지니어 및 기획자
1일	Kube-virt 설명 Libvirt와 가상머신과 관계 이미지 빌드 도구 및 빌드 방식 CI/CD와 가상머신 관계 가상머신 이미지 쿠버네티스 가상머신으로 배포

4. IaC기반 대규모 서비스 배포

교육명	IaC기반 대규모 서비스 배포
교육개요	IaC도구를 사용하여 다수 서버에 서비스를 배포한다. 어떠한 IaC도구가 대용량 서비스 배포에 적합한지, 적합하지 않는 경우 어떠한 방식으로 해당 문제를 해결할 수 있는지 확인한다.
목표	IaC 도구를 통해서 대용량 서비스를 구성 및 배포한다. 대용량 서비스 구성 및 배포를 위해서 어떠한 도구가 적절한지 확인한다. Git Ops를 활용한 IaC도구 활용 방법 그리고 도구에 대해서 확인한다,
기간	1일
코드	SEM-010
교육대상	모든 소프트웨어/시스템 엔지니어 및 기획자
1일	IaC도구 소개 - Ansible-core, community - OpenTofu(Terraform) - Pulumi - Salt 대규모 서버 처리를 위한 IaC 도구 선택 - 왜 쉘 스크립트는 사용하지 않는가? - SSH VS AGENT - 병렬처리 및 분산처리 - 예제 시나리오 살펴보기 랩 - 간단한 플레이북 간단하게 작성하기

	- 도구별 플레이북 실행 결론
--	----------------------------

5. IaC를 위한 AI 부트캠프

교육명	IaC를 위한 AI 부트캠프
교육개요	LLM 기반으로 서버 운영 시, 안전한 작업을 위해서 IaC 도구, 앤서블/오픈도푸와 같은 도구를 통해서 안전하게 작업 수행이 되어야 한다. 어떠한 방식으로 구축 및 운영 해야지 안전한 운영 방식인지 구축 및 확인 해보도록 한다.
목표	인프라 자동화를 위해서 간단한 LocalAI를 구성 및 구축 후 시스템을 코드 작성 및 배포를 수행한다. CPU기반으로 AI 운영에 대해서 간단하게 구축 및 운영 해보도록 한다 간단하게 코드 구성 후 시스템 운영 및 IaC를 LLM기반으로 구성 및 운영한다
기간	2일
코드	BOOT-IAC-LLM
교육대상	모든 소프트웨어/시스템 엔지니어 및 기획자
1/2일	CPU LLM소개 - GPU VS CPU LLM 차이점 - CPU 병렬 LLM 가능한가? - 양자화 그리고 GUFF 포맷 - 시스템 엔지니어 입장에서 ML/AI 그리고 BigData LLM 구성 및 구현 - LLM 설치 및 간단한 실험 운영 - Go언어 기반으로 코드 작성 및 활용 - 왜 파이썬을 활용하지 않는가? - 최종 모델 완성 간단하게 IaC 도구 소개 - IaC 도구에 대한 설명 - Ansible/OpenTofu/Salt - 어떠한 도구가 적절한 도구인가? - 간단한 문법 학습 랩 - 모델 기반으로 간단한 코드 생성 - LLM 기반으로 작업 상태 확인

6. Podman 기반 LLM 서비스 부트캠프

교육명	Podman 기반으로 LLM 서비스 부트캠프
교육개요	오픈소스 기반 LLM 기반으로 어떻게 LLM 서비스를 구성 및 통합하는 방법에 대해서 학습한다.

목표	간단하게 LLM를 구성 후, 어떠한 방식으로 인프라 시스템과 통합이 가능한지 아이디어 스케치 및 직접 구성한다.
기간	1~2일
코드	BOOT-LLM-ENGINE
교육대상	모든 소프트웨어/시스템 엔지니어 및 기획자
1~2일	컨테이너 런타임 소개 - Podman/Docker - 왜 서비스는 POD 기반으로 구성하는가? - LLM과 샌드박스(Sandbox) 개념 - CPU/GPU 기반의 LLM 서비스 차이 - Podman 기반으로 간단한 LLM 서비스 구현 LLM 구축 및 응용 - 시스템 활용을 위한 컨셉 - LLM를 활용한 KIKI 인프라 프로젝트(운영체제/플랫폼) - LLM Engine과 LLM Model - Huggingface와 그리고 RAG - 소규모 사용이 가능한 LLM 서비스 구성(2/3b) 프롬프트 및 에이전트 구성 - LLM 활용을 위한 간단한 에이전트 소개 - LLM의 프롬프트 역할 - 파이선 기반으로 간단한 에이전트 구성 인프라 활용 - KIKI 코드의 지원 범위 및 역할 제한 - 플랫폼 코드 작성 방법(ansible/openstack/kubernetes) - LLM BASH ENHANCEMENT - 로그 분석(실시간 아님) - 사용자 추적 기록 분석

7. Kubeflow 기반 LLM & MLOps 1일 부트캠프

교육명	Kubeflow 기반 LLM & MLOps 1일 부트캠프
수정	2026-02-06
교육개요	본 과정은 Kubernetes 환경에서 Kubeflow를 활용한 LLM 운영 및 MLOps 구조를 하루 만에 이해할 수 있도록 설계된 실습 중심 부트캠프입니다. Podman 기반 단일 노드 LLM 운영과 대비하여, 조직 단위에서 AI 모델을 어떻게 관리·자동화·운영하는지에 초점을 맞춥니다. LLM을 "실행해보는 수준"을 넘어, MLOps 파이프라인 관점에서 바라보는 시야를 확보하는 것을 목표로 합니다.
목표	Kubeflow 기반 MLOps 전체 흐름 이해

	LLM 학습-배포-운영 구조 파악 Podman 기반 LLM과 Kubeflow 기반 운영 방식 비교 이해 조직 단위 AI 플랫폼의 기본 설계 개념 습득
기간	5일
랩	오픈스택 기반, 인터넷 사용 가능한 노트북 요구
교육대상	이론+데모+핵심 실습 Kubeflow 중심 Hands-on "구축"보다는 이해와 흐름 중심
1일	MLOps 개념 및 필요성 Podman 기반 LLM과 MLOps 구조 비교 Kubeflow 전체 구성요소 Pipeline/Training/Serving 역할 LLM 학습-배포 흐름 이해 모델 Artifact 개념 Pipeline 구조 및 YAML 이해 예제 Pipeline 실행 자동화-운영 시 고려사항 On-Prem AI 플랫폼 구조 정리

8. Podman 기반 On-Prem LLM 구축 실무 1-Day 부트캠프

교육명	Podman 기반 On-Prem LLM 구축 실무 1-Day 부트캠프
수정	2026-02-06
교육개요	본 과정은 Kubernetes나 복잡한 MLOps 플랫폼 없이도 LLM을 직접 구축하고 서비스로 운영하는 방법 을 학습하는 실습 중심 과정입니다. Podman 기반 컨테이너 환경에서 LLM Runtime을 구성하고, API 형태로 모델을 서비스화함으로써 소규모 조직·개인·PoC 환경에 적합한 LLM 운영 방식 을 다룹니다. "AI 플랫폼 이전 단계" 에서 현실적으로 가장 빠르게 적용 가능한 LLM 구축 방식 을 목표로 합니다.
목표	Podman 기반 LLM 실행 환경 이해 LLM Runtime 구성 및 서비스화 GPU 없이 가능한 LLM 운영 구조 이해 온프레미스 LLM PoC 아키텍처 설계 역량 확보
기간	
랩	이론+실습 병행 Podman 기반 Hands-on 단일 노드 또는 소규모 환경 기준
교육대상	인프라 엔지니어/시스템 엔지니어 Kubernetes 도입 전 LLM PoC를 원하는 조직

	폐쇄망/온프레미스 환경에서 LLM이 필요한 담당자 AI 모델 "운영 입문자"
1일	LLM 서비스 구조 이해 Public LLM vs On-Prem LLM 비교 Podman 컨테이너 구조 Docker 대비 Podman 차이점 LLM Inference 구조 CPU 기반 LLM 실행 Podman으로 LLM 컨테이너 실행 REST API 형태로 서비스화 로그, 리소스 사용량 확인 실무 적용 시 고려사항 정리